

Curso de
PROGRAMACIÓN DE
PROCESOS Y
SERVICIOS

Unidad
didáctica **01**

PROCESOS

OBJETIVOS

Mediante el estudio de este módulo, pretendemos conseguir los siguientes objetivos:

- Entender qué son los procesos y cuáles son sus características.
- Visión de los sistemas operativos multiproceso.
- Visión general de las diferencias que puede haber entre la ejecución de procesos en distintos sistemas operativos.
- Conocer el modelado de sistemas multiproceso.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



INTRODUCCIÓN

Cualquiera de los dispositivos informáticos que hoy en día utilizamos, bien sea una Tablet, un ordenador personal o un móvil están implementados con un sistema operativo multiprogramado. Los **sistemas operativos multiprogramados** están basados en el concepto de proceso. Un proceso es una secuencia de instrucciones programadas en un lenguaje dado.

Los sistemas operativos multiprogramados tienen la tarea de entremezclar la ejecución de un número de procesos determinado de forma que todos se estén ejecutando de manera simultánea. En los sistemas de tiempo compartido se debe proporcionar además al usuario un tiempo de respuesta razonable de modo que creemos en él la sensación de que todos sus procesos se están ejecutando realmente a la vez y le dan respuesta en un tiempo razonable.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



Concepto que se basa en gestión de **la ejecución simultánea de varios procesos.**

El sistema operativo pues debe asignar una serie de **recursos** a los procesos siguiendo una determinada política, y dependerá de cada sistema operativo y que suele estar basada en prioridades, de manera que todos los procesos puedan llegar a obtener los recursos que necesitan para poderse ejecutar impidiendo el sistema operativo además los interbloqueos entre los procesos. El sistema operativo además debe dar soporte a un sistema de comunicación entre los procesos de manera que puedan ejecutarse de manera simultánea y colaborando entre ellos.



INTRODUCCIÓN

Así pues **el proceso** es la base de los sistemas operativos actuales y la gestión del procesador o procesadores mediante multiprogramación, es la base para la multitarea de todos nuestros dispositivos actuales. En esta unidad definiremos que es un proceso estudiaremos sus propiedades y veremos como el sistema operativo gestiona múltiples procesos de manera simultánea planificando su ejecución, asignando recursos y evitando los bloqueos mutuos, de manera que veamos que un proceso es una unidad ejecutable que necesita recursos para poder ejecutarse y que por otro lado es una unidad planificable por parte del sistema operativo.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



Concepto que se identifica con **la instancia de ejecución de una porción de código.**

1. PROCESOS

Este módulo se centra en el estudio de los siguientes apartados:

- 1.1. ¿Qué es un proceso?.
- 1.2. Descripción interna de un proceso.
- 1.3. Control de un proceso.
- 1.4. Procesos y Threads.
- 1.5. Redes de Petri

Zona reservada para Recursos gráficos y/o
contenidos teóricos complementarios



1. PROCESOS

1.1. ¿Qué es un proceso?

Habitualmente estamos acostumbrados a utilizar el término programa para referirnos a un elemento ejecutable que está programado en un determinado lenguaje de programación. De ese modo para nosotros WinWord o Firefox por ejemplo, son programas que se ejecutan en nuestro sistema.

Sin embargo es muy importante distinguir entre lo que es un programa y lo que es un proceso.



Un programa es una secuencia de instrucciones escritas en un lenguaje de programación que puede ser un lenguaje interpretado o compilado y que finalmente se ejecutará en nuestro sistema.

Zo



1. PROCESOS

1.1. ¿Qué es un proceso?

Así pues en realidad un programa es un concepto estático que hace referencia al código fuente que alguien o nosotros mismos hemos programado previamente en un entorno de desarrollo.



Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios

Un proceso es una instancia que se está ejecutando de una sección del código de un programa, y qué contiene una serie de elementos para controlarlo desde el sistema operativo cómo puede ser su contador de programa sus estados sus registros de procesador y sus distintos segmentos como el segmento de datos el segmento de pila y el segmento de código.

1. PROCESOS

1.1. ¿Qué es un proceso?

Un proceso es una instancia que se está ejecutando de una sección del código de un programa, y qué contiene una serie de elementos para controlarlo desde el sistema operativo cómo puede ser su contador de programa sus estados sus registros de procesador y sus distintos segmentos como el segmento de datos el segmento de pila y el segmento de código.

Así pues podemos ver que un proceso es un concepto dinámico frente al concepto estático de programa y que además no tiene porqué equivaler en una correspondencia uno a uno con el programa. De hecho veremos que muchos programas cuando los ponemos en ejecución se dividen en varios procesos.

Zona reservada para Recursos gráficos v/o



La paralelización de procesos en el sistema

lleva asociado un intento de optimizar los tiempos de ejecución de las aplicaciones, ejecutando simultáneamente tareas que no tengan dependencias entre sí.

1. PROCESOS

1.1. ¿Qué es un proceso?

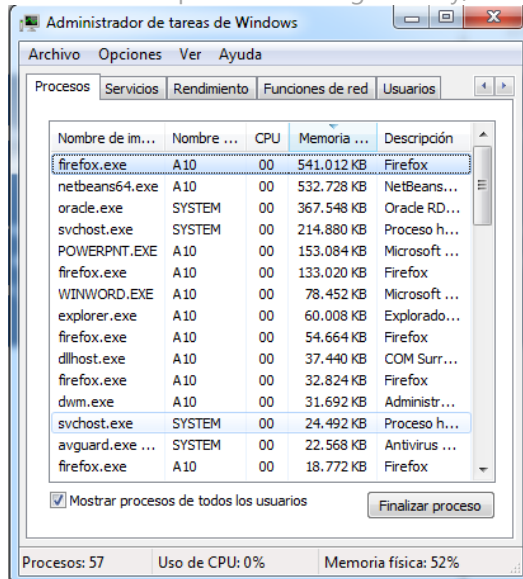
ACTIVIDAD



Entra en el administrador de tareas de tu sistema. En sistemas Windows por ejemplo pulsando **CONTROL + ALT + Suprimir**.

Orden de los procesos por nombre pinchando en la columna Nombre de la pestaña procesos. Acto seguido arranca el Firefox o Google Chrome y comprueba cuántos procesos se han puesto en marcha.

Zona reservada para Recursos gráficos y/o

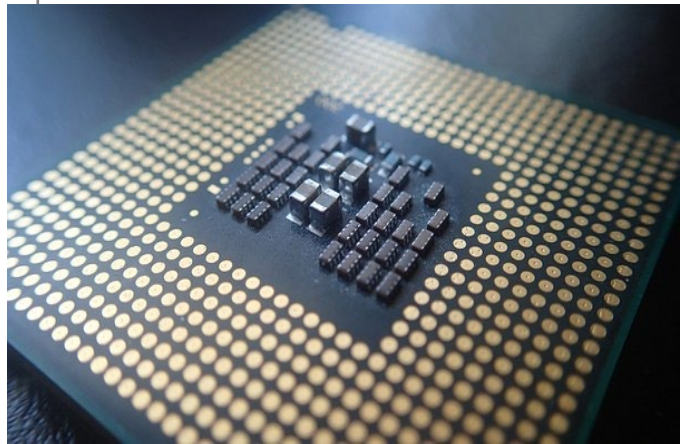


1. PROCESOS

1.1. ¿Qué es un proceso?

Con la actividad anterior nos podemos dar cuenta de que estás correspondencia un programa un proceso no siempre se cumple. De hecho muchos de los programas que utilizamos ponen en ejecución distintos procesos simultáneos cada uno de ellos encargado de una tarea, de este modo se intenta maximizar el rendimiento de las aplicaciones mediante la especialización de sus procesos en tareas concretas. Más adelante veremos también como los procesos compiten por obtener un hueco en el procesador de modo que al dividir un programa en varios procesos también nos puede servir para aumentar la tasa de tiempo que mi programa está ejecutándose dentro del procesador.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



Es interesante también de tenerse en el concepto de que un proceso es una instancia de ejecución de parte del código de un programa, ya que es posible que un mismo programa se ha ejecutado de manera simultánea por varios usuarios en un sistema multiusuario coma de manera que cada una de estas ejecuciones generará al menos un proceso con su propio contador de programa sus propios registros sus propias pilas de datos y de código etcétera.

De esta manera nuestro sistema operativo gestión del procesador para poder implementar la multiprogramación y los sistemas de tiempo compartido mediante este concepto de proceso planificable por lo que nuestros procesos son las unidades mínimas planificables del sistema que necesitan una asignación de procesador para ejecutarse además de otros posibles recursos cómo pueden ser acceso a memoria, ficheros de comunicaciones etcétera.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



LECTURAS ADICIONALES RECOMENDADAS

Proceso según Microsoft:

- <https://msdn.microsoft.com/es-es/communitydocs/win-dev/os/manejador-de-procesos-procesos>

Proceso según Linux:

- https://www.linuxtotal.com.mx/index.php?cont=info_admon_012

1. PROCESOS

1.1. ¿Qué es un proceso?



Sistema multiproceso:

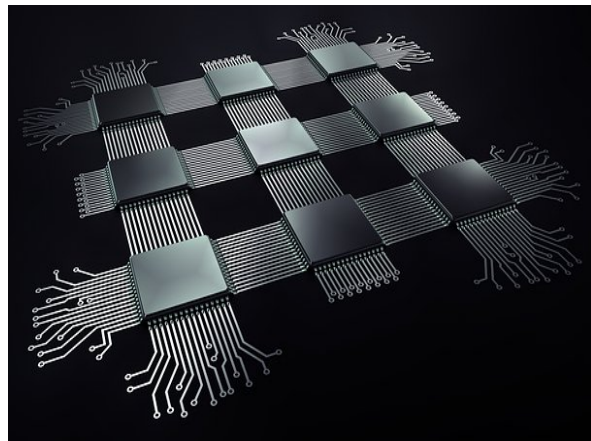
multiproceso

m. adj. INFORM. Modalidad operativa de un sistema informático que permite la ejecución simultánea de varios programas compartiendo la memoria central y las unidades periféricas.

(Fuente: es.thefreedictionary.com/multiproceso)

Básicamente es el concepto que engloba la ejecución de varios procesos simultáneos en un sistema. Así como los sistemas multitarea monoprocesador permiten a varios procesos compartir un solo procesador, varios procesadores pueden ser utilizados para ejecutar múltiples procesos que pertenecen a un único proceso.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



1. PROCESOS

1.1. ¿Qué es un proceso?

EJEMPLO DE APLICACIÓN.

Para comprender mejor este concepto básico de lo que es un sistema operativo multiproceso, vamos a tomar como ejemplo, de manera distendida para hacer una comparativa con una de las atracciones que en la ciudad de Zaragoza tenemos cuando se acercan las Fiestas del Pilar.

Se trata del denominado Tragachicos, que básicamente es una figura gigante que en su interior contiene un tobogán, de manera que los chicos entran por la boca del gigante y salen deslizándose por su digamos ... “salida estándar”.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



1. PROCESOS

1.1. ¿Qué es un proceso?

EJEMPLO DE APLICACIÓN.

Por supuesto y como en cualquier fiesta popular se generan largas colas de chicos que quieren subir a la atracción. Además justo en la escalera que da acceso a la boca del Tragachicos siempre hay uno de los voluntarios del Ayuntamiento que ayudan a organizarse y planificar la subida de los chicos.

En este caso y para hacernos idea muy gráfica de lo que puede ser un sistema multiproceso con planificación FIFO (es decir first in first out) podríamos identificar a cada uno de los chicos que quiere bajar por el tobogán como si fuese uno de los procesos que compete por el recurso estrella de nuestro sistema que es el procesador. Por supuesto los voluntarios que organizan la subida y planificación de los niños serían el Scheduler o planificador.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios

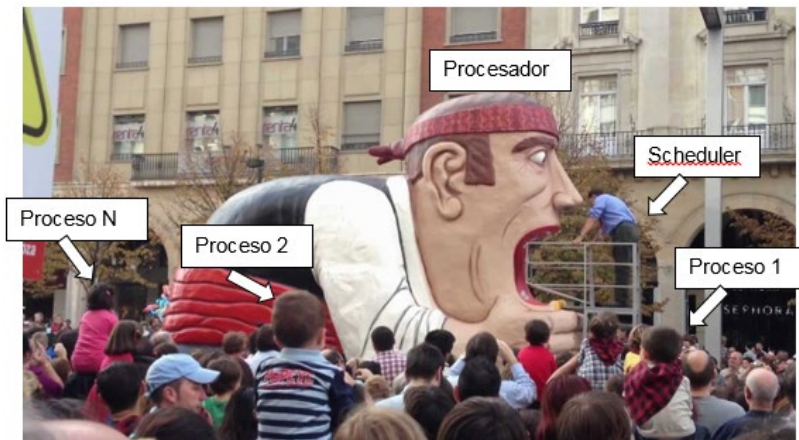


Figura 1.2. Sistema Multiproceso con planificación tipo FIFO

1. PROCESOS

1.1. ¿Qué es un proceso?

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



El planificador (en inglés scheduler) es un componente funcional muy importante de los sistemas operativos multitarea y multiproceso, y es esencial en los sistemas operativos de tiempo real. Su función consiste en repartir el tiempo disponible de un microprocesador entre todos los procesos que están disponibles para su ejecución.

(Fuente Wikipedia)

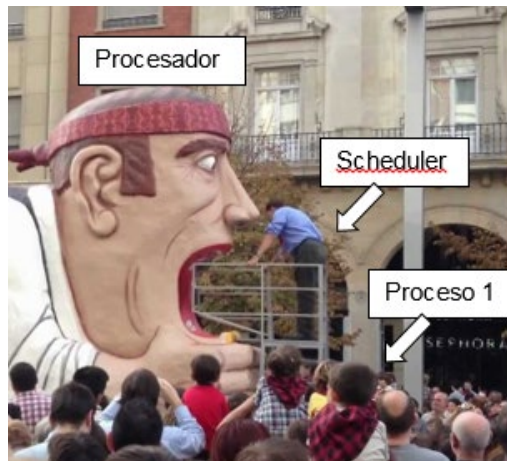


1. PROCESOS

1.1. ¿Qué es un proceso?

El Scheduler es pues el Daemon, o programa de sistema residente del núcleo del sistema operativo que se encarga de gestionar y planificar los procesos para que todos puedan ejecutarse. En este sistema multiproceso se ha elegido el sistema FIFO como planificación de procesos es decir el primer niño que llega será el primer niño que suba la escalera y se ha tragado por el Tragachicos (baje por el tobogán), pero existen otros sistemas multiproceso en los que a lo mejor a procesos críticos les dan prioridad de manera que podríamos pensar que si en este sistema del Tragachicos hubiese algún proceso con prioridad serían procesos que se cuelan al resto de procesos y no guarda en la fila como todos los demás.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



En ningún caso nuestro sistema aunque estuviera basado en prioridades debe permitir que **ningún proceso muera de inanición** es decir que ningún proceso se quede eternamente esperando su porción de ejecución en el procesador porque hay otros procesos más prioritarios que él, de manera que nuestro sistema garantiza que aunque con prioridad baja todos los procesos entra en ejecutarse al menos una vez en uno de los ciclo planificables del procesador.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



1. PROCESOS

1.1. ¿Qué es un proceso?

Si miramos ahora de nuevo la actividad anterior en la que analizábamos como un programa se puede dividir en varios procesos, ahora se puede entender mejor el porqué muchos programas utilizan esta **técnica de multiprogramación**.

Imaginemos que un Papa va con su niño al Tragachicos, este sería el caso en el que un programa genera un único proceso de ejecución, mientras que otro niño va con su papá, su mamá y su abuelita, este sería el caso en que un programa al ejecutarse genera tres procesos.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



1. PROCESOS

1.1. ¿Qué es un proceso?

Mientras en uno de los ciclos completos del procesador el niño que va con un solo papá baja una vez por el tobogán y se vuelve a poner a la fila, el niño que va con su papá su mamá y su abuelita tiene la ventaja de que, mientras el niño está con el papá en la fila, la abuelita se pone a hacer fila también pero un poquito más atrás y la mamá asimismo se pone hacer fila también un poquito más atrás que la abuelita.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



1. PROCESOS

1.1. ¿Qué es un proceso?

De esta manera, cuando niño baja la primera vez por el Tragachicos, luego en vez de volverse a poner al final de la fila, busca a su abuelita que ya estaba guardando el sitio en la fila, de manera que adelanta al niño que iba solo con su papá (y está pacientemente esperando al final de la fila) y baja una segunda vez por el Tragachicos. Pero la cosa no termina ahí, sino que también lo puede llegar a hacer una tercera vez, ya que cuando baje por segunda vez por el Tragachicos gracias al puesto que le guardaba su abuelita en la fila, irá a buscar a su mamá, que también estará en posiciones adelantadas de la fila.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



1. PROCESOS

1.1. ¿Qué es un proceso?

Con esto lo que logramos es que en un ciclo completo de planificación de procesos de nuestro procesador, mientras un programa está entrando sólo una vez porque sólo tiene un proceso, el otro programa está entrando tres veces ya que tiene tres procesos ejecutándose de manera simultánea y el sistema les dará su porción de procesador, con lo que conseguimos que el programa de tres procesos estadísticamente vaya tres veces más rápido en ejecución que el programa que solo tiene un proceso.

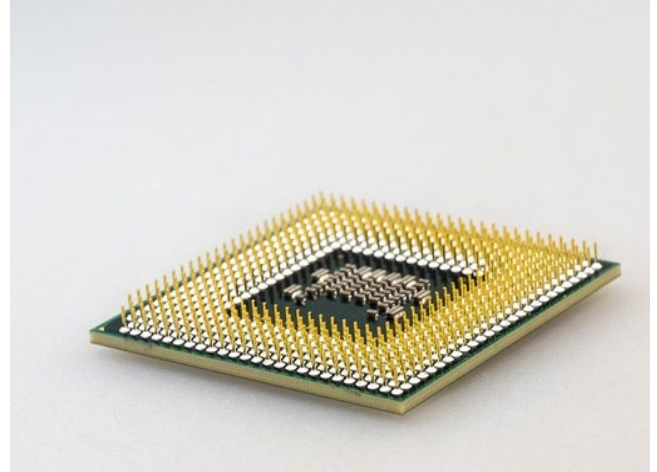


Ahora que conoces lo que es un sistema multiproceso y un sistema de planificación de procesos (FIFO), investiga qué otros sistemas de planificación de procesos existen en los sistemas operativos actuales.

Cada vez que se crea un proceso en el sistema, bien porque se pone en ejecución un programa o una sección de código de ese programa, este proceso pasa por una serie de estados que definen su comportamiento, estos estados son

- En ejecución
- Preparado
- Bloqueado

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



Estados de los procesos

- En ejecución: cuando un proceso está en ejecución está ocupando de manera efectiva el procesador es decir se está ejecutando.
- Preparado: un proceso preparado es aquel que estando fuera del procesador, dispone de todos los recursos necesarios para poderse ejecutar y únicamente necesita un tiempo de ejecución en el procesador para continuar su ejecución.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



Estados de los procesos

- **Bloqueado:** un proceso bloqueado está fuera del procesador y es un proceso que necesita algún recurso para poder continuar su ejecución además del propio procesador. Estos recursos que necesita el proceso pueden ser un dispositivo, un dato etcétera. Mientras esos recursos no estén disponibles el proceso seguirá bloqueado y cuando uno de esos recursos se libere o esté disponible el sistema operativo mandará un evento o alarma al proceso que está esperando el recurso de manera que sepa que su recurso esperado ya está disponible y pueda continuar con su ejecución pasando a la lista de procesos preparados cuando disponga ya de todos los recursos necesarios salvo el procesador.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



Estados de los procesos

- Existe otra serie de estados extendidos que veremos más adelante pero por sencillez en esta primera aproximación consideraremos únicamente estos tres estados y la ejecución en un sistema monoprocesador, de modo que solo podemos tener un proceso en ejecución a la vez. A su vez pueden existir diversos procesos en el estado de preparados y también varios procesos pueden estar en el estado de bloqueados como de manera que se forman listas que ordenan esa salida de procesos de cada uno de los estados.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



- Esas listas en el caso de procesos preparados no siempre tienen porqué ser listas ordenadas, de manera que pueden ser listas basadas en prioridades. Los procesos de sistema y de aplicaciones críticas suelen tener prioridades altas de manera que tienen más quantum de reloj que el resto de procesos, es decir se ejecutan más veces por segundo que el resto de procesos menos prioritarios.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



1. PROCESOS

1.1.1 Estados de un proceso y Transiciones de estado de los procesos



Definición

El Quantum de reloj es la porción de tiempo de procesador que se le concede a cada uno de los procesos como máximo para ejecutarse. Se puede dar el caso de que una ejecución llegue a solicitar al sistema un recurso o una operación de Entrada Salida, por lo que en ese momento el proceso será expulsado del procesador sin haber consumido el total de su quantum de reloj. Terminado el Quantum del proceso y aunque este no haya terminado de ejecutarse es expulsado del procesador para dejar paso a otro proceso.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



1. PROCESOS

1.1.1 Estados de un proceso y Transiciones de estado de los procesos

La lista de procesos bloqueados por otra parte en general no es una lista ordenada ya que los procesos no se desbloquearán por orden de prioridad sino que se desbloquearan en función de recibir los eventos que les notifican que los recursos que necesitan ya están disponibles. En ese momento como hemos visto pasan a la lista de procesos preparados. Existen situaciones peligrosas que pueden llevar a que un proceso uno esté esperando recursos que tiene asignados al procesados y a su vez al proceso dos estoy esperando recursos que tiene asignados el proceso 1 de manera que ninguno de los dos procesos puede continuar para ejecutarse ya que ambos están en un bucle de espera mutua. Esta paradoja puede llevar a interbloqueo de procesos y la veremos más adelante en el ejemplo clásico de los filósofos pensantes.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



1. PROCESOS

1.1.1 Estados de un proceso y Transiciones de estado de los procesos

Existen varias **políticas de planificación de procesos** que intentan evitar estos bloqueos mutuos entre otras se puede asignar prioridades también a los procesos bloqueados y como último caso el kernel elige al azar a uno de los procesos en los que ha detectado un interbloqueo y termina con el de manera que los recursos bloqueados por eso proceso quedan disponibles para el resto de procesos.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios





El núcleo del Sistema Operativo (en inglés kernel) es un componente funcional muy importante de los sistemas operativos, y consta de una serie de procesos de sistema que controlan totalmente el sistema, entre estos procesos de sistema se encuentra el Scheduler.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



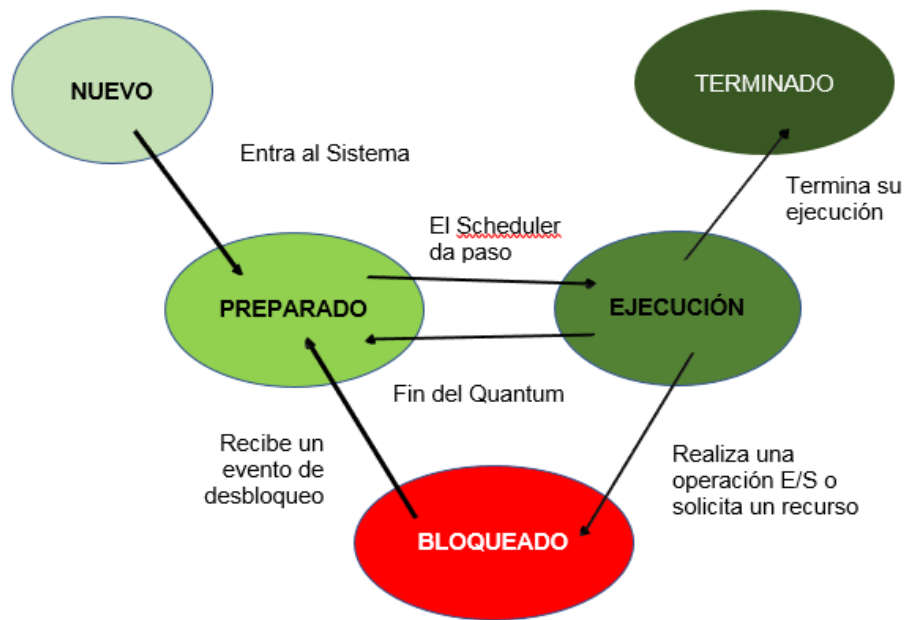
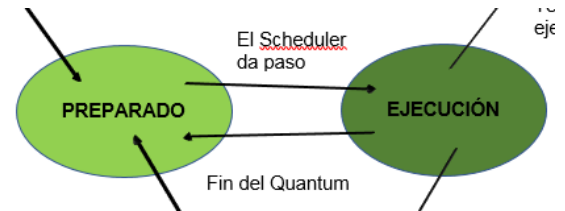


Figura 1.3. Los estados simplificados de un proceso

En la figura anterior podemos ver los ejemplos de transición de un estado a otro que sufren los procesos

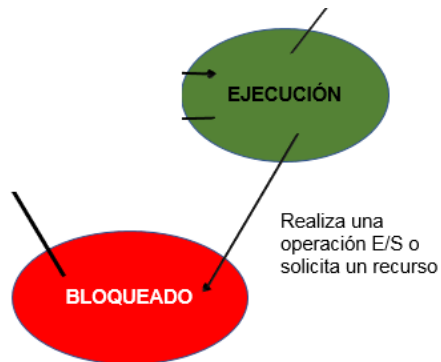
- Del Estado de preparado a ejecución: Cuando un proceso se pone en marcha entra directamente al estado de procesos preparados para ejecutarse. De momento no ha solicitado ningún recurso ya que no ha podido ejecutar ni una sola de sus líneas de código por lo que no estará bloqueado. Es entonces cuando el Scheduler del sistema operativo va dando paso a los procesos para que entren al procesador a ejecutarse de uno en uno de manera que cuando sea el turno de nuestro proceso (el Scheduler le da paso) entrará a ejecución en el procesador.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



- **Del estado de ejecución a bloqueado:** nuestro proceso se está ejecutando en el procesador llevará un contador de programa que le indica que línea de código le toca ejecutar en cada momento y en un determinado momento realiza una operación de entrada salida o una operación de espera basada en un semáforo de acceso a una zona de memoria exclusiva o cualquier otra operación que solicite un recurso. Es en este momento cuando el sistema operativo se da cuenta de que nuestro proceso aunque tenga quantum de reloj para salir ejecutándose no puede hacerlo por falta de recursos por lo que lo desplaza al estado de procesos bloqueados y guarda su contexto para que cuando retome su ejecución el proceso puede salir ejecutándose en el mismo punto exacto y con los mismos valores en sus registros como cuando se estaba ejecutando en el procesador y fue expulsado de él.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



1. PROCESOS

1.1.1 Estados de un proceso y Transiciones de estado de los procesos

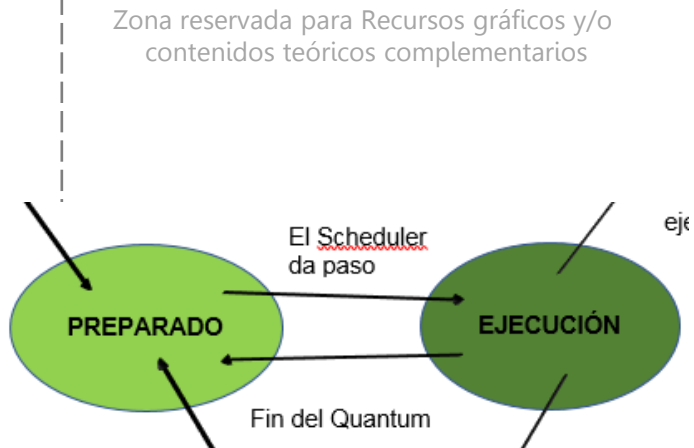
Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



Atención

El tiempo que el usuario utiliza para operaciones de entrada salida como por ejemplo dar a un proceso datos a través del teclado, supone para el proceso una auténtica eternidad en sus ciclos de ejecución de manera que no puede estar desperdiciando ciclos de CPU a la espera de que el usuario teclee sus datos. Es por esto que el teclado tiene un buffer de memoria en el que va almacenando las letras tecleadas y periódicamente las envía a la memoria interna del ordenador y es entonces cuando el sistema operativo avisa de que ha terminado la operación de entrada salida entre el usuario y el proceso desbloqueando sea el proceso y pudiendo contar a partir de entonces con los datos que el usuario le ha proporcionado.

- **De ejecución a preparado:** en este caso estamos en un sistema de tiempo compartido y el proceso ha consumido todo su tiempo de ejecución en el procesador es decir ha agotado su quantum y quiere seguir ejecutándose porque no necesita ningún otro recurso que no sea el propio procesador. pero como agotado el tiempo máximo asignado de ejecución debe liberar el procesador para dejar paso a otro proceso de modo que es expulsado del procesador y se guarda todo su contexto al igual que en el caso anterior coma su contador de programa, sus registros de datos etc para que cuando retomé la ejecución pueda seguir ejecutándose en el mismo punto y con los mismos valores en sus registros que cuando fue expulsado del procesador.



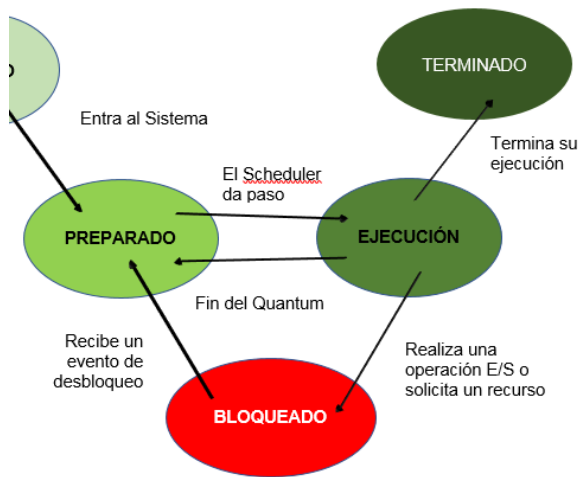
1. PROCESOS

1.1.1 Estados de un proceso y Transiciones de estado de los procesos

- **De ejecución a terminado:** en este caso el proceso ha terminado su ejecución de manera que realiza una llamada determinación al sistema operativo que se encarga de liberar todos los recursos que tenía asignados incluyendo las zonas de memoria que ocupaba el proceso.

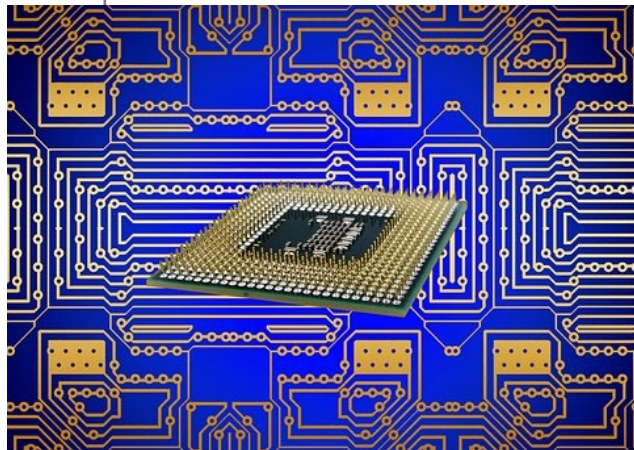


- **De desbloqueado a preparado:** En este caso un proceso que estaba esperando a que uno de los recursos que necesitaba, ha recibido una notificación por parte del sistema operativo de que ya está disponible ese recurso que necesitaba. En este caso el proceso vuelve al estado de preparados y se pone en la fila a esperar que le toque su quantum de procesador para poder seguir ejecutándose.

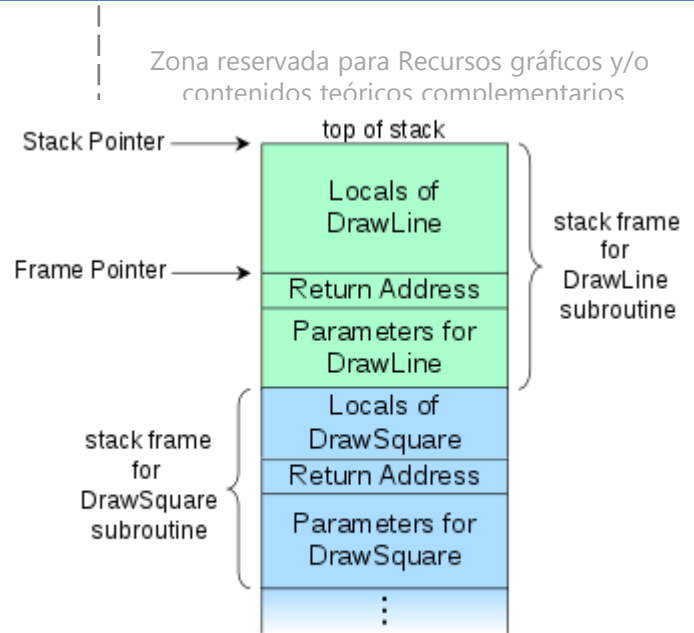


- Como hemos visto un proceso es un concepto dinámico de una porción de código de un programa en ejecución ahora bien ¿qué contiene un proceso en su interior?.
- Un proceso está formado en su interior por una serie de bloques. Partiendo de los bloques mínimos, podemos definir el bloque que almacena el código que tiene que ejecutar y el bloque que contiene los datos que utiliza el proceso.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



- **Habitualmente los procesos también contienen un bloque de pila** en el que se almacenan las llamadas a funciones y procedimientos y los parámetros que se intercambian entre ellos así como los resultados de las llamadas a procedimiento o función. Finalmente existe un bloque que contiene una serie de atributos del proceso para que el sistema operativo lo pueda gestionar, este es bloque de control de proceso o PCB (Process Control Block).

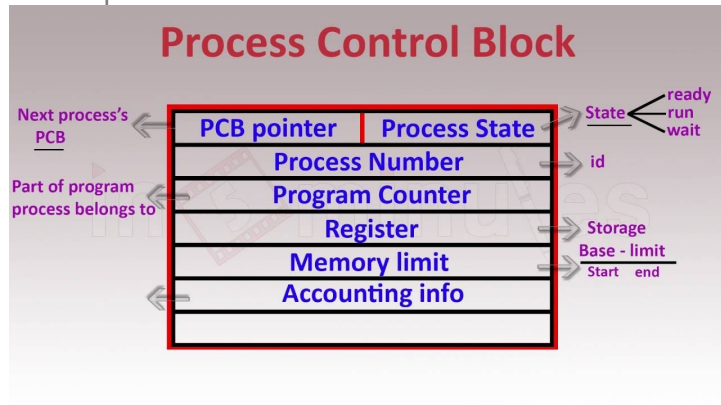


1. PROCESOS

1.2.1. El bloque de control de proceso (PCB).

- **El bloque de control de proceso** es la estructura que contiene los atributos de ejecución de nuestro proceso y que utiliza el sistema operativo para poder gestionar a cada uno de nuestros procesos. Únicamente el sistema operativo tiene acceso a estos bloques de control para realizar tareas de planificación tratamiento de interrupciones asignación de recursos y análisis y supervisión de cada uno de sus procesos.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



1. PROCESOS

1.2.1. El bloque de control de proceso (PCB).

- Todos estos bloques de control de proceso los almacena el sistema operativo en **una tabla de procesos** residente en memoria principal, para facilitarle el acceso a ella a alta velocidad debido a la frecuencia con que se consulta. Dependiendo del tipo de sistema operativo esta tabla está organizada de una manera u otra de manera que se puede implementar cómo lista enlazada o como vector.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios

```
1:25am up 3:25, 1 user, load average: 0.00, 0.00, 0.00
52 processes: 51 sleeping, 1 running, 0 zombie, 0 stopped
CPU states: 0.0% user, 9.0% system, 0.0% nice, 90.9% idle
Mem: 126844K av, 44864K used, 81980K free, 37840K shrd, 2092K buff
Swap: 136512K av, 0K used, 136512K free 27032K cached
```

PID	USER	PRI	NI	SIZE	RSS	SHARE	STAT	LIB	%CPU	%MEM	TIME	COMMAND
1195	vivek	19	0	860	860	668	R	0	9.2	0.6	0:00	top
1	root	0	0	476	476	404	S	0	0.0	0.3	0:04	init
2	root	0	0	0	0	0	SW	0	0.0	0.0	0:00	kflushd
3	root	0	0	0	0	0	SW	0	0.0	0.0	0:00	kupdate
4	root	0	0	0	0	0	SW	0	0.0	0.0	0:00	kpiod
5	root	0	0	0	0	0	SW	0	0.0	0.0	0:00	kswapd
6	root	-20	-20	0	0	0	SW<	0	0.0	0.0	0:00	mdrecoveryd
408	bin	0	0	428	428	340	S	0	0.0	0.3	0:00	portmap
423	root	0	0	0	0	0	SW	0	0.0	0.0	0:00	lockd
424	root	0	0	0	0	0	SW	0	0.0	0.0	0:00	rpciod
433	root	0	0	560	560	472	S	0	0.0	0.4	0:00	rpc.statd
447	root	0	0	476	476	412	S	0	0.0	0.3	0:00	apmd
498	root	0	0	528	528	428	S	0	0.0	0.4	0:00	syslogd
507	root	0	0	764	764	388	S	0	0.0	0.6	0:00	klogd
521	nobody	0	0	624	624	520	S	0	0.0	0.4	0:00	identd
525	nobody	0	0	624	624	520	S	0	0.0	0.4	0:00	identd

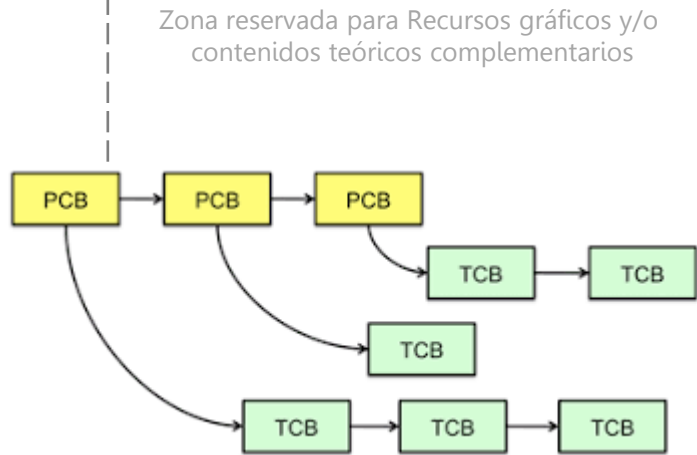
La información que guarda el bloque de control de proceso se puede clasificar en:

- **Datos para la identificación del proceso**
- **Datos sobre el estado del proceso**
- **Datos para el control y gestión del proceso**



- **Datos para la identificación del proceso**

En estos datos está el PID del proceso, en ocasiones habrá casos en los que se permite a que un proceso crea otro nuevo proceso por lo que también entre estos atributos se encuentran identificadores que permiten enlazar el proceso creador con los procesos creados. También entre esta información de control del proceso se encuentra el UID del propietario del proceso.



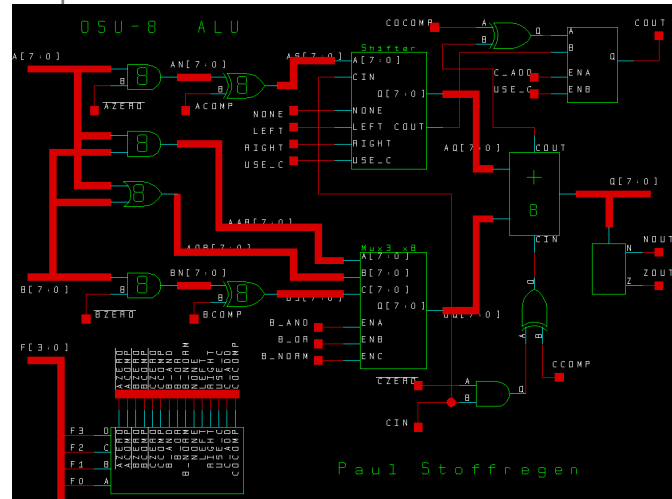
1. PROCESOS

1.2.1. El bloque de control de proceso (PCB).

- Datos sobre el estado del proceso**

Esta información es principalmente el contenido de los registros del procesador en tiempo de ejecución del proceso. Contiene el PC y todos los registros de datos del procesador, incluyendo los registros de control de Estado y los punteros a pilas. De modo que mientras el proceso está en modo de ejecución estos valores se encuentran en los registros del procesador pero cuando el proceso va a ser expulsado del procesador bien porque se le ha terminado su quantum de reloj o porque ha recibido una interrupción el sistema operativo debe garantizar que se salvaguarda toda la información de los registros del procesador incluyendo el PC para que si en un momento posterior el proceso vuelve al procesador pueda reanudar su ejecución en el punto exacto de ejecución en el que estaba y con los mismos valores en los registros que tenía.

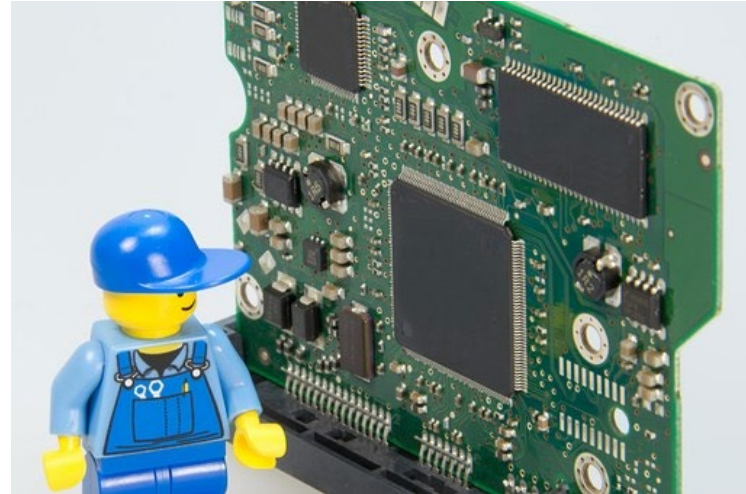
Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios

- **Datos para el control y gestión del proceso**

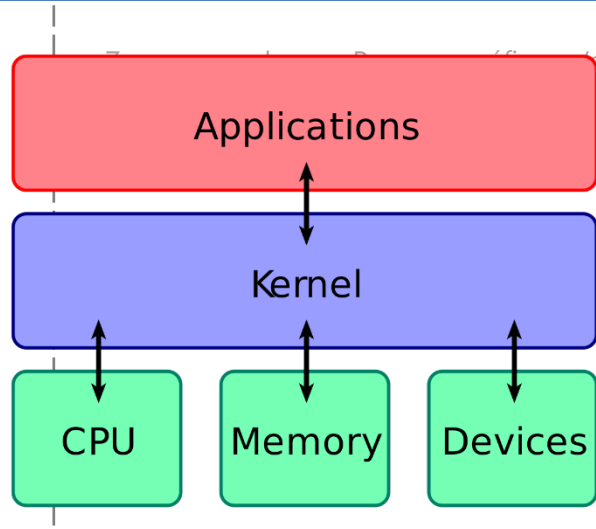
Aquí se encuentra toda la información que nuestro sistema operativo utilizará para controlar y coordinar a todos los procesos activos en el sistema como por ejemplo información sobre el estado y planificación del proceso como por ejemplo su prioridad y el estado de proceso (si está listo para ejecutarse o está bloqueado esperando un recurso etcétera). También incluye punteros a diversas estructuras de datos y zonas de memoria que utiliza el proceso así como los recursos utilizados por el proceso como pueden ser ficheros abiertos, ficheros de comunicación etcétera



1. PROCESOS

1.2.1. El bloque de control de proceso (PCB).

De esta manera **nuestro kernel** es capaz de crear nuevos procesos y eliminar suspender y reanudar a los procesos existentes, también puede cambiar la prioridad de un proceso para subirlo en la cola de procesos preparados y que no muera de inanición y bloquear o desbloquear procesos a través de interrupciones de sistema. Nuestro Carmelo con toda esta información también será capaz de planificar la entrada ejecución de nuestros procesos asignándoles procesador y permitir que los procesos se comuniquen entre ellos como veremos en temas posteriores.



El Kernel o núcleo del Sistema Operativo es el conjunto de rutinas de sistema que se ejecutan en modo privilegiado y que se encarga del control de todo nuestro sistema operativo, responsabilizándose del acceso seguro al hardware y de gestionar recursos a través de llamadas al sistema.

Cuando un proceso se elimina del sistema se borra su entrada en la tabla de procesos y su PCB, por lo que sus recursos quedan liberados para poder ser reasignados. Toda la memoria que este proceso ocupaba queda asimismo disponible. La eliminación de un proceso sin embargo en la mayoría de los sistemas no implica que se destruyan los hijos que este ha creado por lo que cada uno de los procesos mantienen su independencia y persistencia. Encuentra la suspensión o bloqueo de procesos y su reanudación puede ser debida a distintos motivos.

Zona reservada para Recursos gráficos v/o



Download from
Dreamstime.com

This watermark-free image is for previewing purposes only.



122367392

Aguirre | Dreamstime.com

Como hemos visto un proceso se puede bloquear porque ha solicitado un recurso no disponible pero también el sistema operativo puede activar la suspensión de ciertos procesos en puntos de carga máxima del sistema para así reducir la carga de procesos que entran a ejecución. La reanudación de cada uno de los procesos depende por supuesto de su prioridad y de la ocupación de memoria y tiempo de procesador del sistema. Reanudar un proceso implica que se pone a ese proceso en la cola de procesos preparados en el mismo estado que tenía cuándo fue suspendido.

Zona reservada para Recursos gráficos v/o



Download from
Dreamstime.com

This watermark-free image is for previewing purposes only.

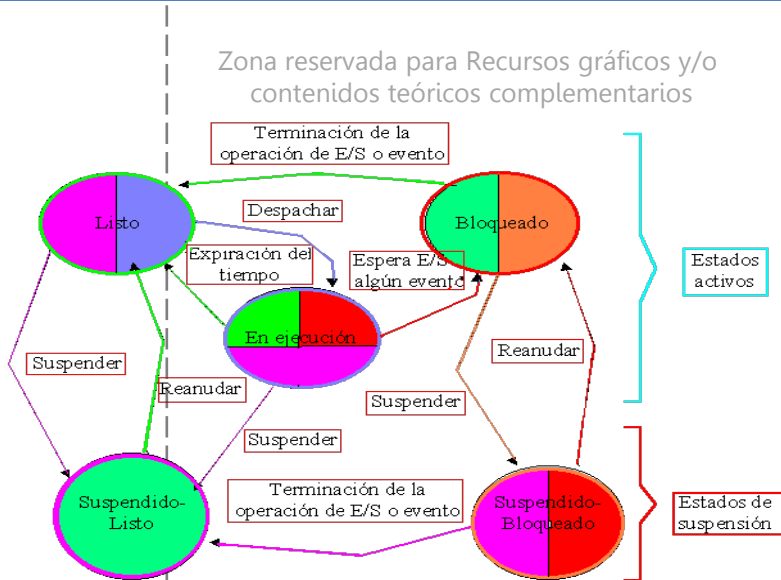


122367392

Aguirre | Dreamstime.com

La figura siguiente muestra un **diagrama extendido de transiciones de estado** para nuestros procesos.

Podemos observar que tenemos un par de estados extra que son los estados de suspensión. Estos estados de suspensión pueden ser activados por el propio sistema operativo o incluso por el propio proceso cuando detecta que necesita recursos sabiendo que no están disponibles de modo que el propio proceso se puede enviar a sí mismo a un estado de suspensión a la espera de que alguien le avise cuando el recurso necesario esté ya disponible. En sistemas multiprocesador podemos encontrar incluso el caso de que un proceso de sistema que esté ejecutándose en un procesador suspenda a un proceso que se está ejecutando simultáneamente en otro procesador.



1.4 Los estados de un proceso

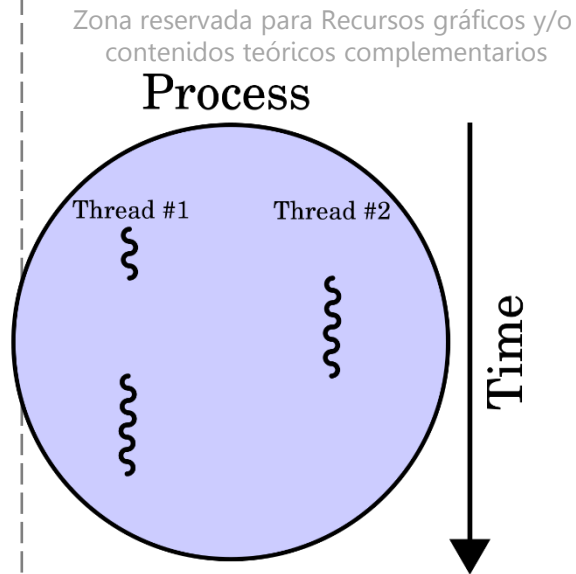
El cambio de proceso es la operación que realiza el scheduler del sistema operativo cuándo es necesario sacar a un proceso del procesador y dejar pasar a ejecución al siguiente proceso del bloque de procesos preparados.

Este cambio del proceso como hemos visto anteriormente se puede deber a diversos eventos que suceden en el sistema. Dependiendo del evento que haya sucedido veremos si es necesario hacer un cambio de contexto entre procesos y si el sistema operativo tiene que realizar alguna acción con las estructuras de datos que gestionan los procesos.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios

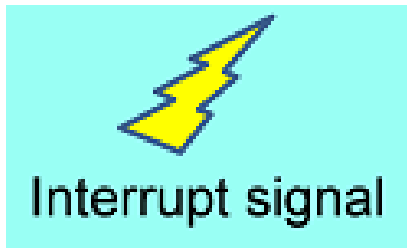


Básicamente un **cambio de proceso** de un proceso puede generarse por una interrupción de sistema. Existen distintos tipos de interrupciones de sistema un grupo de ellas indican que sucedido algo externo al proceso que se está ejecutando como puede ser una operación de entrada salida o una señal que le ha enviado otro proceso.



El segundo gran grupo de interrupciones de sistema está relacionado con las excepciones o errores asociados al proceso en ejecución como pueden ser intentos de acceso a zonas prohibidas de memoria, divisiones entre cero o errores en tiempo de ejecución. El sistema operativo cuándo ocurre una interrupción transfieren el control al gestor de interrupciones que identifica el tipo de interrupción ocurrida y dispara la rutina del sistema operativo que maneja la interrupción que se ha producido.

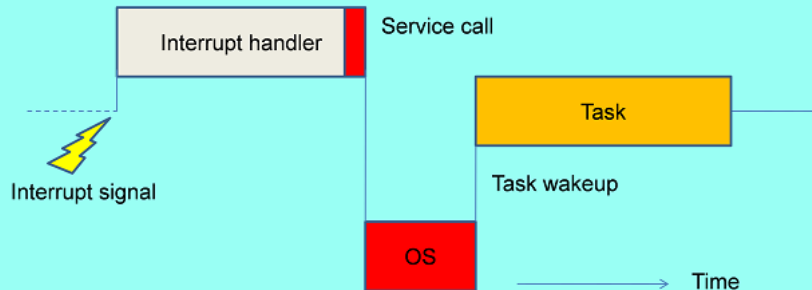
Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios





El manejador (en [inglés](#) handler) es un componente funcional de los [sistemas operativos](#) que se encarga de atrapar y controlar las excepciones. Se trata de procesos de sistema residentes en memoria que son rutinas programadas para dar respuesta a los casos de excepción identificándolos y ejecutando una porción de código que sea capaz de dar respuesta al evento sucedido.

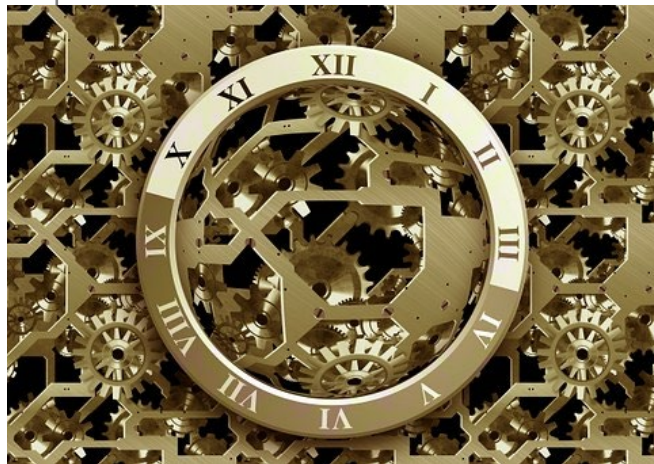
■ Flow from an Interrupt to Task Switching



Podemos ver ahora **algunas de estas interrupciones** qué son:

- **Interrupción de reloj:** se genera cuando el proceso ha consumido su quantum de reloj asignado punto en Sistemas Operativos de tiempo compartido es la base para la multiprogramación dando un tiempo de respuesta aceptable al usuario. en este caso el proceso cambiará su estado de ejecución a preparado y se asignará el procesador a un nuevo proceso que elegir a el Scheduler.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



- **Interrupción de Entrada/Salida:** En este caso el sistema operativo ha detectado una operación de entrada salida. El sistema operativo discrimina si esta operación es un evento que estaba esperado uno o varios procesos, y caso de ser así avisa a todos los procesos que estaban bloqueados qué elemento ha sucedido y los traslada al grupo de procesos preparados para ejecución. En ese momento los procesos competirán por el procesador y cuando pase uno de ellos ha estado de ejecución podrá reservar el recurso que necesitaba para continuar ejecutándose.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios

Memory

Job 1
Waiting

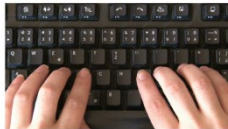
Job 2
Ready

Job 4
Ready

CPU



Interrupt



- Interrupción por fallo de página:** esta interrupción sucede cuando un proceso intenta acceder a una dirección de memoria que no está paginada todavía en nuestra memoria es decir es una dirección de memoria que hacía tiempo que no había sido utilizada por lo que está almacenada en memoria secundaria. En realidad se trata como si fuese una interrupción de entrada salida ya que en ese momento el sistema a través de sus mecanismos de paginación traerá de memoria secundaria memoria principal la página entera de datos que el proceso necesita para seguir trabajando. Ya que esta operación conlleva muchos ciclos de procesador para que el proceso no esté consumiendo ciclos de procesador de manera ociosa, el sistema operativo realiza un cambio de contexto y pone al proceso que se estaba ejecutando en la lista de procesos bloqueados punto cuando la página se haya cargado en memoria principal el proceso volverá a reanudarse pasando al estado de preparados.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



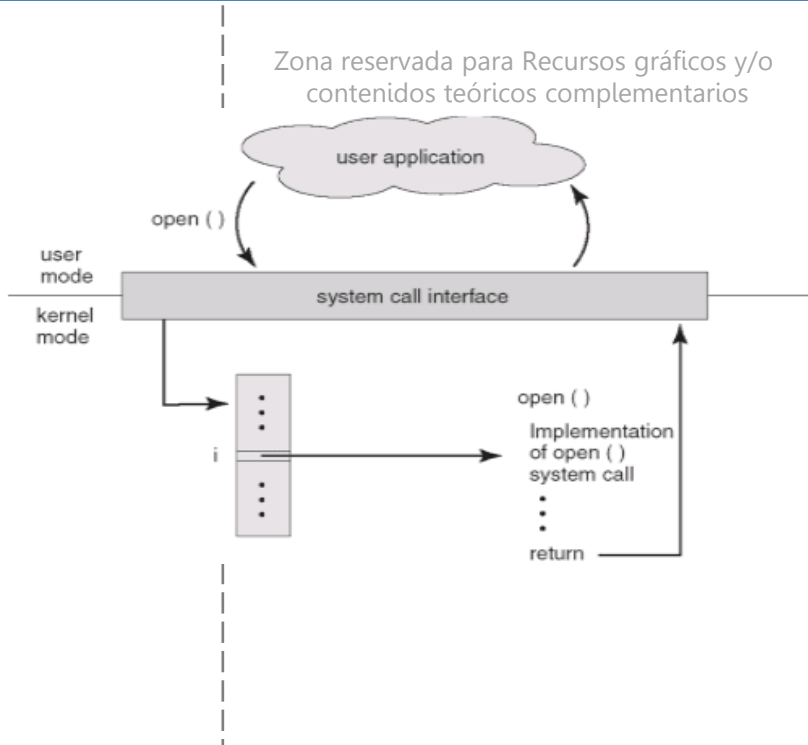
1.3.1 Cambio de proceso.

- También pueden dar interrupciones debidas a **errores fatales**. En ese caso el proceso que se está ejecutando es eliminado del sistema y no se guarda su contexto, dándole Scheduler paso al siguiente proceso que está en el estado de preparados.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios

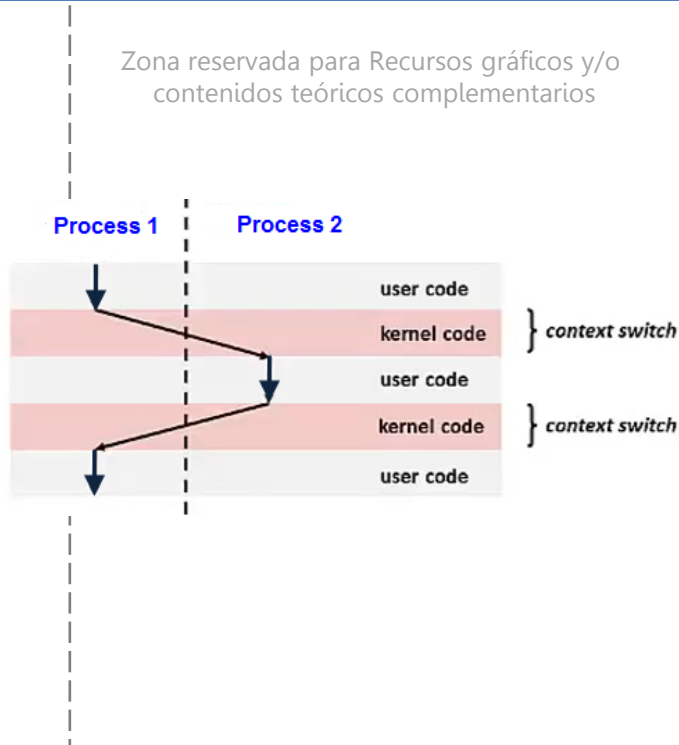


Los procesos que se encuentran en ejecución también pueden **ejecutar interrupciones llamando a rutinas del sistema** operativo, como por ejemplo cuando un proceso necesita abrir un fichero, en ese momento el proceso solicita el sistema operativo una operación de entrada salida lo que llevará a que el proceso quede bloqueado hasta que entre un proceso de sistema que de servicio a su petición de apertura de fichero y le devuelvan los parámetros necesarios para controlar ese fichero en ese momento el proceso bloqueado pasará al estado de preparados y en su pila de invocación a la rutina de sistema el proceso de sistema que ha abierto el fichero le habrá apilado los parámetros que necesita para poder gestionar el fichero abierto.

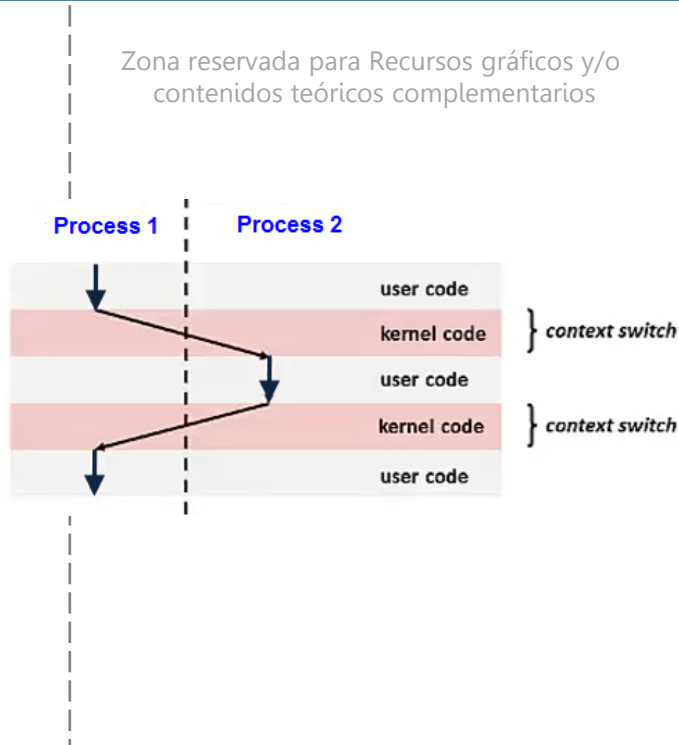


Cada vez que tengamos la necesidad de sacar a un proceso del procesador no porque haya terminado sino por alguna interrupción necesitaremos salvar su contexto, para poderlo cargar de nuevo en el momento que el proceso quiera volver a entrar a ejecutarse al procesador.

El contexto de un proceso es básicamente el pc, los registros del procesador y la información de pila, además de toda la información necesaria para poder volver a poner en marcha el proceso, por ejemplo si nuestra interrupción fue provocada por una operación de entrada salida habrá que almacenar también las condiciones de error que manejan el gestor de interrupciones del sistema operativo.



Por supuesto **al realizar un cambio de contexto** hay que actualizar el PCB que estaba en estado de ejecución y actualizarlo con el nuevo estado. El sistema operativo tendrá además que añadir el PCB a la lista del nuevo estado del proceso. Acto seguido el Scheduler nos dará un nuevo proceso para ejecutar de manera que tendremos que actualizar el procesador con los datos de ese nuevo proceso y cambiar el PCB del nuevo proceso poniendo su estado en ejecución.



Todo este cambio de contexto por supuesto implica que estamos invirtiendo para poderlo realizar una serie de ciclos de procesador, por lo tanto podemos contabilizar que el ejecutar en paralelo dos procesos con sus correspondientes cambios de contexto va a costarnos más ciclos de reloj es decir más tiempo que se ejecutan amo secuencialmente los dos procesos pero en cargas mayores de procesos estadísticamente los procesos terminan antes su ejecución que si se ejecutarán de manera secuencial.

Para dejar claro este concepto veremos un ejemplo y realizaremos la siguiente actividad.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



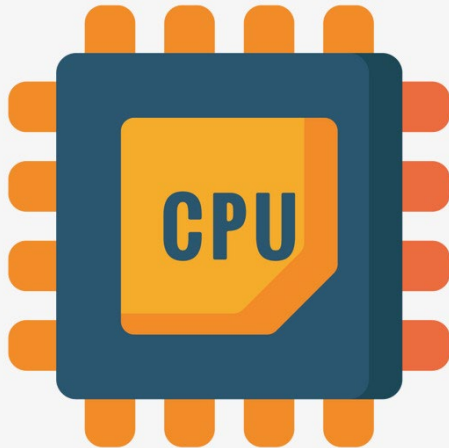
Ejemplo de distribución de procesos en un sistema multiproceso

Tomemos por simplificar el ejemplo un sistema monoprocesador en el que el quantum de reloj lo vamos a identificar como una unidad de tiempo. Tenemos además seis procesos en el sistema, mismos uno de ellos necesitará un tiempo distinto de ejecución.

De ese modo nuestros procesos son:

Proceso	Tiempo necesario para ejecutarse
P1	7
P2	5
P3	3
P4	4
P5	2
P6	3

Zona reservada para Recursos gráficos y/o



Ejemplo de distribución de procesos en un sistema multiproceso

Si tuviésemos un sistema operativo que realiza una organización secuencial de los procesos la ejecución sería cómo sigue:

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios

Proceso	Tiempo necesario para ejecutarse	Quantum Procesador																								
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
P1	7	■	■	■	■	■	■	■																		
P2	5								■	■	■	■	■													
P3	3													■	■	■										
P4	4																■	■	■	■						
P5	2																				■	■				
P6	3																						■	■	■	
		■																								
		■																								
		■																								

1.5 Ejemplo de ejecución secuencial de procesos en sistema monoprocesador

Ejemplo de distribución de procesos en un sistema multiproceso

Si tuviésemos un sistema operativo que realiza una organización secuencial de los procesos la ejecución sería cómo sigue:

En este escenario vemos que los tiempos de finalización de los procesos son:

Proceso	Tiempo necesario para ejecutarse	Tiempo de finalización
P1	7	7
P2	5	12
P3	3	15
P4	4	19
P5	2	21
P6	3	24

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios

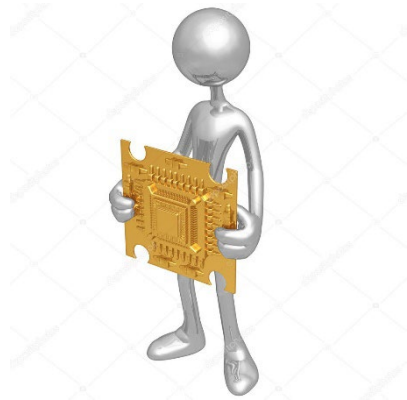
		Quantum Procesador																								
Proceso	Tiempo necesario para ejecutarse	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
P1	7																									
P2	5																									
P3	3																									
P4	4																									
P5	2																									
P6	3																									

Inicio de la ejecución del proceso
 Proceso ejecutándose
 Fin de la ejecución del proceso

Ejemplo de distribución de procesos en un sistema multiproceso

Desde luego **en un sistema de tiempo real**, el ejecutar los procesos con esta política secuencial sería muy frustrante para los usuarios de los procesos tardíos que están esperando respuesta. Es por este motivo que aparece el concepto de sistema multiprogramado, en el que la ejecución de los procesos se va intercalando en el procesador repartiendo los quantum de reloj del mismo entre los procesos a ejecutar.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



1.6 Ejemplo de ejecución multiproceso en sistema monoprocesador

Ejemplo de distribución de procesos en un sistema multiproceso

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios

En este escenario vemos que los tiempos de finalización de los procesos son:

Proceso	Tiempo necesario para ejecutarse	Tiempo de finalización
P1	7	24
P2	5	22
P3	3	15
P4	4	20
P5	2	11
P6	3	17

1. PROCESOS

1.3.2 Cambio de contexto de un proceso.

Ejemplo de distribución de procesos en un sistema multiproceso

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios

Por lo tanto teniendo en cuenta que el cambio de contexto frente al quantum de reloj es una cantidad de tiempo despreciable, podemos darnos cuenta de que los procesos estadísticamente terminan en los tiempos similares a los de la programación secuencial salvo el caso del proceso 1 que es el máximo perjudicado del cambio de política de planificación de procesos.

Sin embargo es mucho más importante el fijarnos en los posibles tiempos de respuesta al usuario, ya que en un sistema multiprogramado el proceso dos por ejemplo podría estar dando respuesta al usuario ya en el quantum de procesador número 2 mientras que en una planificación secuencial no daría respuesta al usuario hasta como pronto el quantum 8.

[illegible]

1. PROCESOS

1.3.2 Cambio de contexto de un proceso.

Ejemplo de distribución de procesos en un sistema multiproceso

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios

Teniendo en cuenta el proceso 6 en la planificación secuencial no estaría dando respuesta al usuario hasta el instante 22 mientras que en la planificación multiprogramado ya estaría ejecutándose de manera efectiva en el quantum 6, por lo tanto de cara al usuario en un sistema de tiempo real esta planificación es muchísimo más recomendable, ya que de otra manera correremos el riesgo de que el usuario piense que los procesos que ha lanzado a ejecutar han sufrido algún problema y se han quedado colgados por lo que normalmente los abortará y reiniciará obteniendo idénticos resultados.

Es por esto que los sistemas operativos actuales se utiliza este concepto de multiproceso.

[illegible]



Actividad

Ahora que conoces cómo se gestionan los Quants de reloj de un procesador, realiza una planificación de los siguientes procesos indicando en qué instante de tiempo terminaría cada uno de ellos en un entorno de planificación secuencial y en otro de planificación multiproceso. Analiza los tiempos de terminación de los procesos señalando cuáles son los más beneficiados y los más perjudicados, haciendo además una valoración global del sistema para ver si en este escenario mejora o empeora la ejecución efectiva de los procesos.

Proceso	Tiempo necesario para ejecutarse	Tiempo de finalización
P1	7	24
P2	5	22
P3	3	15
P4	4	20
P5	2	11
P6	3	17

Como hemos visto en un proceso es una porción de código en ejecución es decir es un flujo de ejecución de una parte de un programa que además puede interactuar con otros procesos intercambiándose datos y compartiendo recursos. Así pues un proceso contiene dos grandes áreas la primera la correspondiente a la ejecución del propio proceso y la segunda correspondiente a la gestión de recursos que el proceso necesita para poder ejecutarse.

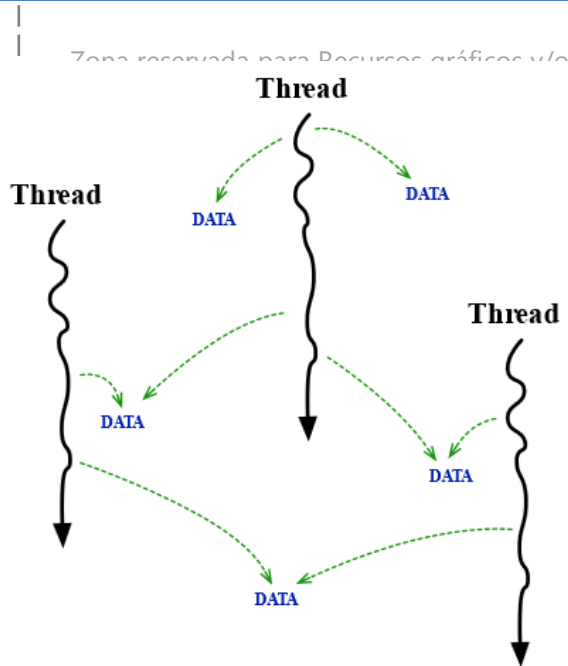
Estas dos grandes áreas del proceso se pueden tratar de forma independiente de manera que muchos de los sistemas operativos actuales aprovechan las características de un proceso a la hora de poner en ejecución nuevos procesos a partir del primero. Aparece así el concepto de thread o hilo de ejecución.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios

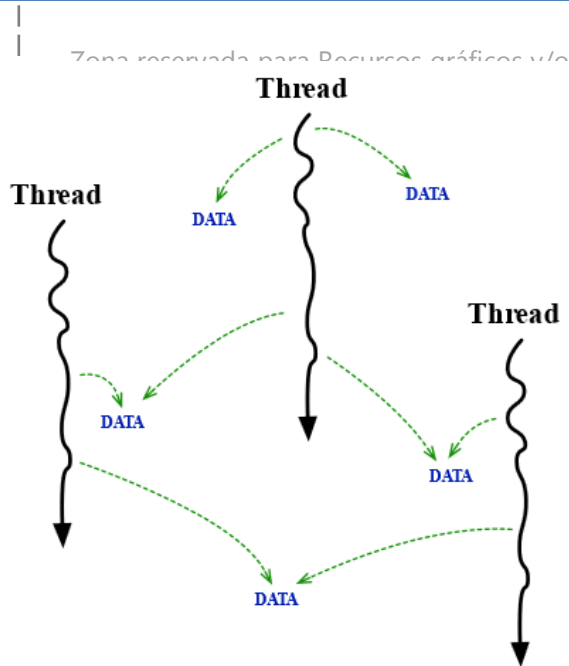


Un hilo de ejecución (en [inglés](#) thread) es un concepto identificable con un proceso, ejecuta una porción de código de mi programa y gestiona una serie de recursos para poder llevar a cabo su ejecución.

Básicamente los lenguajes de programación actuales y los sistemas operativos modernos implementar la ejecución de procesos a través de hilos para mejorar el rendimiento del sistema ya que es más sencillo crear un hilo a partir de un de un proceso que ya existe, y sus cambios de contexto también son más veloces.

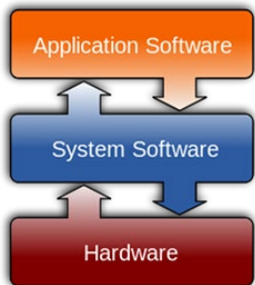


También a la hora de terminar un hilo se ahorra tiempo ya que en todos estos procesos lo que estamos haciendo es mantener un proceso base con varios flujos de ejecución de modo que lo único que mantenemos en memoria es una única copia del código del proceso que compartirán todos sus hilos de ejecución, ya que la zona de memoria es una zona de solo lectura y ninguno de los hilos la podrá modificar, de modo que todos los hilos pueden estar referenciando a la misma copia de sección de código sin necesidad de estar consumiendo una memoria propia cada uno de los hilos.

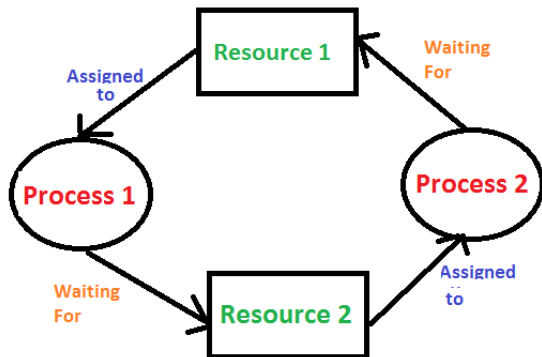


Por supuesto cada uno de nuestros hilos gestionará independientemente sus recursos y podrá haber competencia por ellos.

Este problema lo veremos en las siguientes unidades y es uno de los puntos clave en los sistemas multiprogramados, ya que si no lo gestionamos correctamente podemos caer en bloqueos mutuos, es decir esperas circulares por recursos de los procesos que pueden llevar bloquear mi sistema.

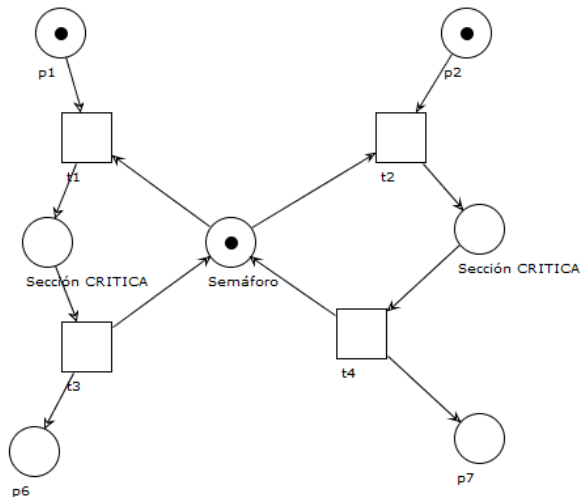


Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



Las redes de Petri son una herramienta que nos sirven para modelar sistemas en los que cambiamos de un estado a otro por causa de un evento. Nos permiten además desde un estado poder llegar a diversos estados disparando diversos eventos y viceversa, poder llegar a un estado porque se ha disparado un evento concreto de los varios posibles que llevaban a ese estado destino.

Zona reservada para Recursos gráficos v/o

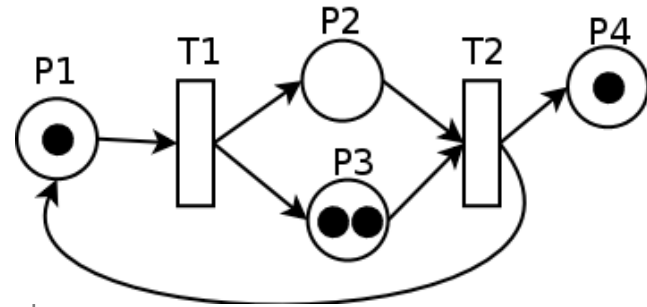




Una Red de Petri es una representación matemática o gráfica de un sistema a eventos discretos en el cual se puede describir la topología de un sistema distribuido, paralelo o concurrente. La red de Petri esencial fue definida en la década de los años 1960 por Carl Adam Petri. Son una generalización de la teoría de autómatas que permite expresar un sistema a eventos concurrentes.

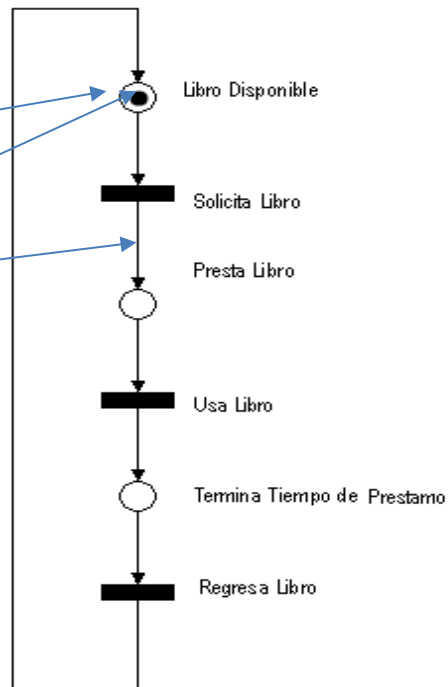
(Fuente: Wikipedia)

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios

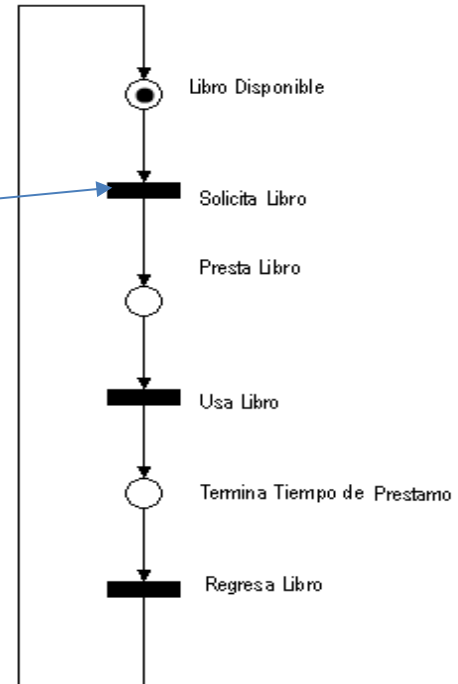


Las redes de Petri cuentan básicamente con cuatro elementos que son :

- **El Estado:** Se representa por un círculo y tiene asociado un nombre descriptivo.
- **Los arcos de direccionalidad:** Se representan por flechas y llevan de un estado a una transición o de una transición a un estado.
- **Los tokens o marcas:** Se representan por un punto (•) y se localizan dentro de los estados, representando así el estado en que se encuentra un proceso



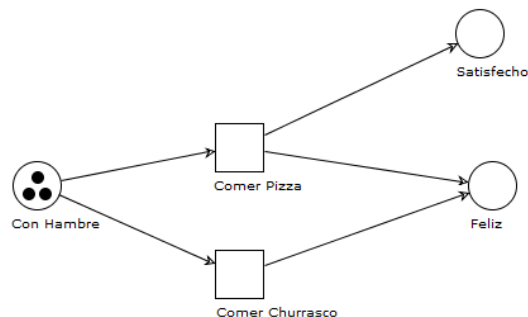
- **Evento o transición:** Se representa por un rectángulo, tiene asociado un nombre descriptivo del evento que representa, y tiene al menos un arco de entrada que viene desde un estado y un arco de salida que va a un estado. Para poder activar una transición debe al menos haber un token en el estado origen de cada uno de los arcos de entrada y se genera un token de salida en cada uno de los estados de salida por cada arco de salida que lleva a ellos.



De este modo, para representar un sistema con una Red de Petri, partimos de un estado inicial en el que al menos debe haber un token y desde el cual soy capaz de disparar una o varias transiciones que me llevarán a uno o varios estados.

Un ejemplo típico podría ser el siguiente en el que se modela a una persona que tiene hambre y como ejecutando una acción de comer algo puede cambiar de estado a otro en el que se sienta feliz o se sienta satisfecho, dependiendo de lo que la persona coma.

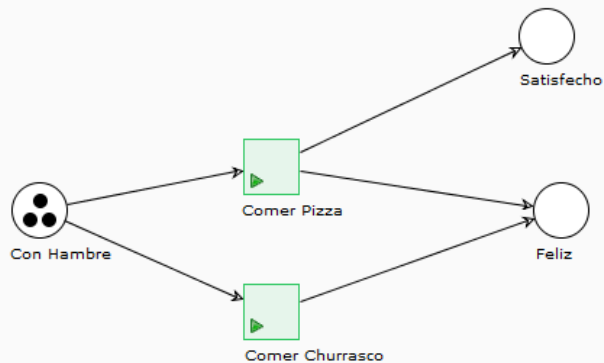
Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



1.6 Ejemplo de Red de Petri: Sistema “Tengo hambre”

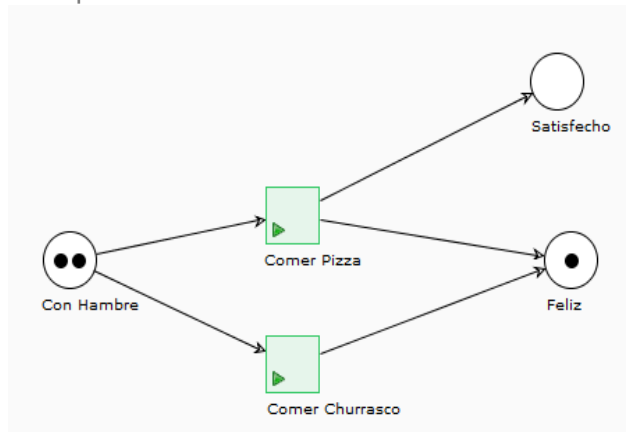
A partir del estado inicial podemos ver que hay representadas tres personas que tienen hambre, desde ese estado se puede disparar para cada una de ellas una de las dos transiciones que consumirán un token del estado “Con hambre” para poderse accionar.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios



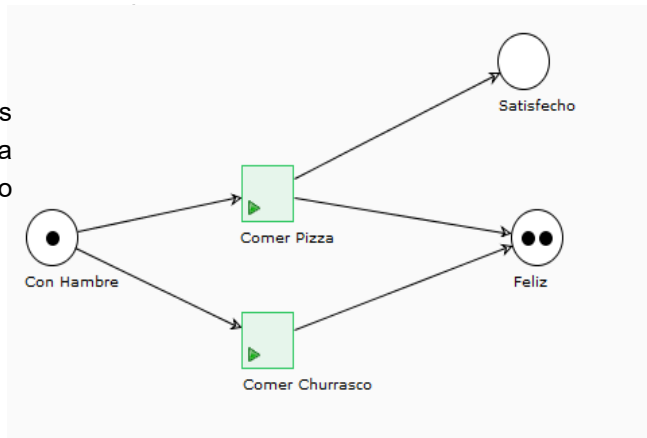
1.7 Ejemplo de Red de Petri: Activación de las transiciones

En el momento que accione por ejemplo la transición inferior (Comer churrasco) se consumirá un token del estado origen y se generará uno nuevo en el estado destino (Feliz), de este modo estoy representando que una de las personas que estaban con hambre ha comido churrasco y ya se siente feliz. Siguen quedando por supuesto todavía dos personas en estado “con hambre”.



1.8.Cambio de estado tras el disparo de la transición “Comer Churrasco”

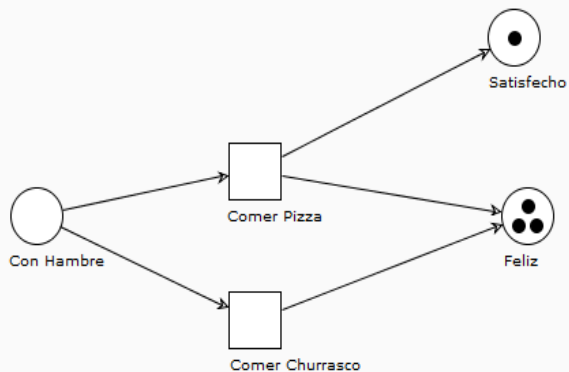
En este momento puedo disparar de nuevo una de las dos transiciones, de modo que puedo por ejemplo volver a elegir la transición “Comer Churrasco” y sucederá el mismo encadenamiento de sucesos con los tokens



1.9.Cambio de estado tras el disparo por segunda vez de “Comer Churrasco”

Ahora de nuevo puedo elegir una transición, en este último caso elegiré la transición “Comer Pizza” que me llevará a consumir un token del estado origen y generar uno nuevo en el estado Feliz y otro en el estado Satisfecho, de modo que se puede ver que un mismo evento puede generar más de un token de salida creando un resultado de estado compuesto en el que tras comer pizza la persona queda feliz y satisfecha al mismo tiempo.

Es interesante ver también que ya no puedo disparar ninguna transición tras esta última, al no haber ya más token disponibles en el estado origen que era el que alimentaba con tokens a las transiciones.



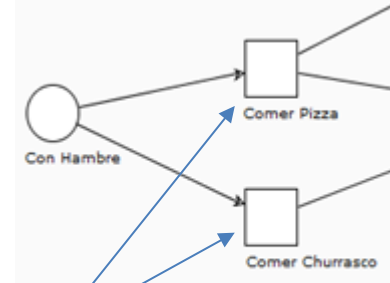
1.10. Cambio de estado tras el disparo por de la transición “Comer Pizza”



Una transición NO se puede disparar sino existen tokens en cada uno de los estados desde los que parten los arcos de entrada a la transición. Al menos debe existir un token por arco de entrada a la transición para que ésta se pueda disparar.

Así pues puede parecer que modelar un sistema con una Red de Petri es un juego de niños, con elementos muy intuitivos y reglas muy básicas, pero veremos más adelante como una Red de Petri con elementos tan simples es capaz de modelar sistemas muy complejos en los que incluyamos desdoblamiento de ejecución de procesos, puntos de reunión de procesos (joins), semáforos que cierran zonas de exclusión mutua etcétera.

Zona reservada para Recursos gráficos y/o
complementarios



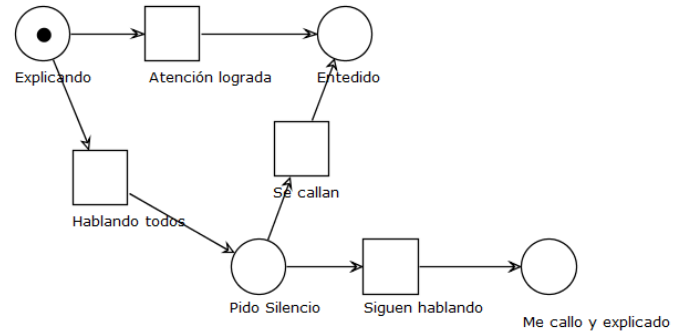
!!! Transiciones inactivas !!!

Ejemplo de Red de Petri: Profesor explicando en clase

Partimos del estado en que un profesor está explicando a partir de ahí tenemos dos posibles transacciones que podemos disparar y que son que el profesor se encuentre con que ha logrado la atención de la clase y la segunda transacción que podemos disparar es que se encuentre el profesor con que aunque él está en el estado de explicando el resto de la clase y están hablando todos entre ellos.

Así pues el profesor podrá disparar en una de las dos transacciones según sea el escenario en el que se encuentre.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios

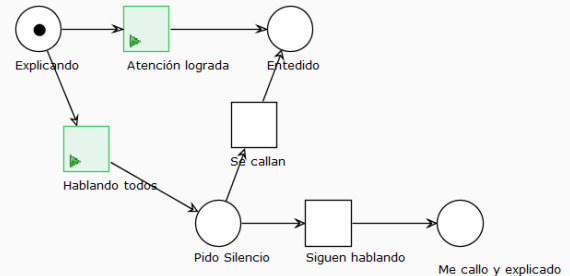


1.11. Red de Petri de profesor explicando en clase

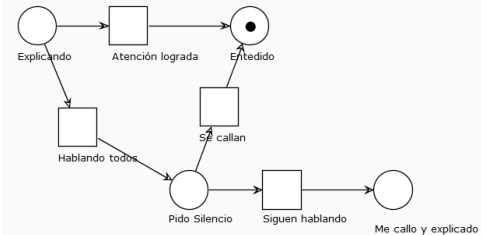
Ejemplo de Red de Petri: Profesor explicando en clase

Si el profesor ve que toda la clase está atenta y decide disparar la transacción de “atención lograda”, llegaremos al estado de que los alumnos han entendido la explicación y daremos por finalizada la ejecución de nuestro sistema, ya que no hay más posibilidades de disparo de ninguna otra transacción llegando a un estado además satisfactorio.

En otro caso deberá disparar la segunda transacción (Hablando todos) de manera que llegar al estado en que necesitará llamar la atención a los alumnos y pedirles silencio.



1.12. Transacciones activas

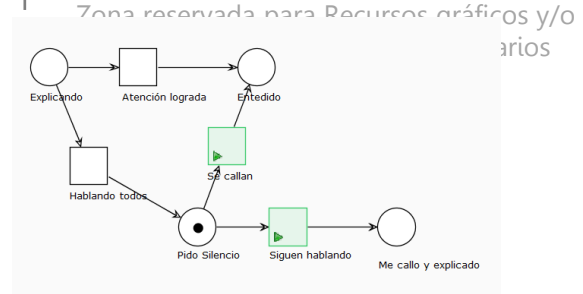


1.13. Disparo de Atención lograda

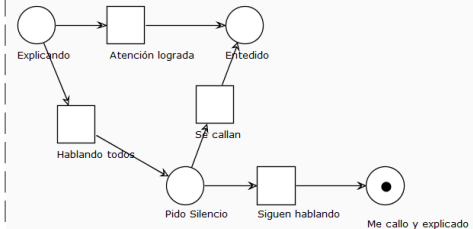
Ejemplo de Red de Petri: Profesor explicando en clase

A partir de este estado de nuevo al que llegamos tras disparar la transacción “Hablando todos”, el profesor puede ver que hay dos transacciones que puede disparar, si petición de silencio que ha realizado es efectiva puede disparar la transacción se calla de manera que puede seguir explicando y llegar al estado de que la lección ha sido entendida por los alumnos.

De otra forma puede disparar la transacción siguen hablando de manera que llegará al estado de me calló y doy todo por explicado qué es otro de los posibles estados finales del sistema aunque en esta ocasión es un estado no satisfactorio.



1.14. Transacciones activas



1.15. Disparo de Siguen Hablando

Concurrencia y sincronización de procesos

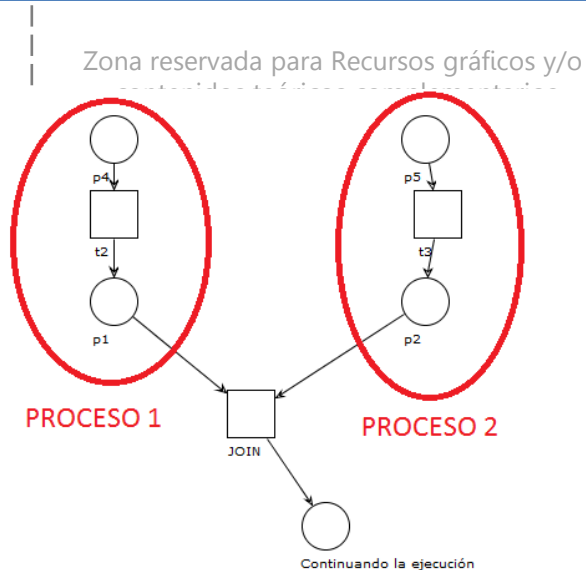
Las redes de Petri son capaces de modelar de manera sencilla sistemas complejos en los que existen relaciones de paralelismo, zonas de exclusión mutua y sincronización de procesos. De este modo, viendo las siguientes estructuras sencillas, podemos llegar a modelar todo tipo de sistemas.

Sincronización de procesos o Join



Definición

Una sincronización de procesos o JOIN, es un punto en el que un proceso debe esperar a otro para que ambos puedan continuar su ejecución.

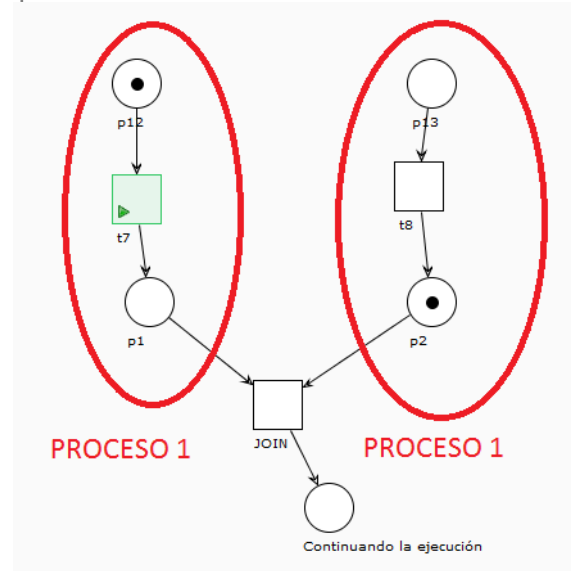


1.16. Join o sincronización de procesos

Sincronización de procesos o Join

Podemos ver claramente como hasta que el proceso 1 no llegue al estado p1 y el proceso 2 no llegue al estado p2 la transacción JOIN no estará activa, ya que necesita de dos tokens para poderse activar.

En el ejemplo vemos como el proceso 2 ha llegado ya al estado p2 pero el proceso 1 todavía no está en el estado p1, de modo que la alimentación de la transacción JOIN no está completa, por lo que se encuentra inactiva y la única manera de que el sistema evolucione es que el proceso 1 active su transacción que llevará a depositar un token en el estado p1 que a su vez activará el JOIN y el sistema podrá continuar ejecutándose, de modo contrario el proceso P2 seguirá esperando al proceso P1 en el JOIN.



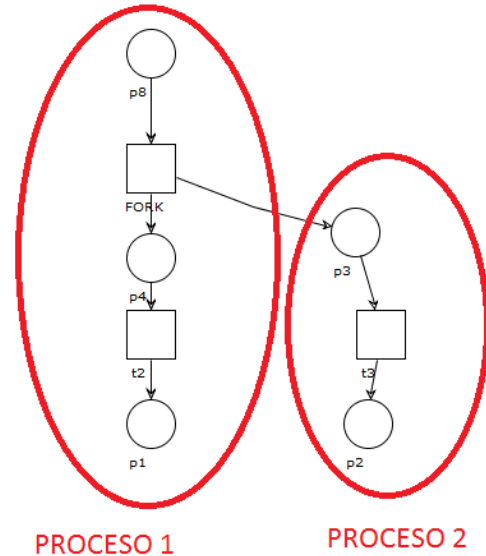
1.17. Join o sincronización de procesos

Creación de procesos o Fork



Una creación de un nuevo proceso o FORK, es un punto en el que un proceso crea a otro proceso para que a partir de ese punto puedan continuar su ejecución de manera independiente y simultánea, de modo que cada uno de ellos pueda ejecutar un determinado código.

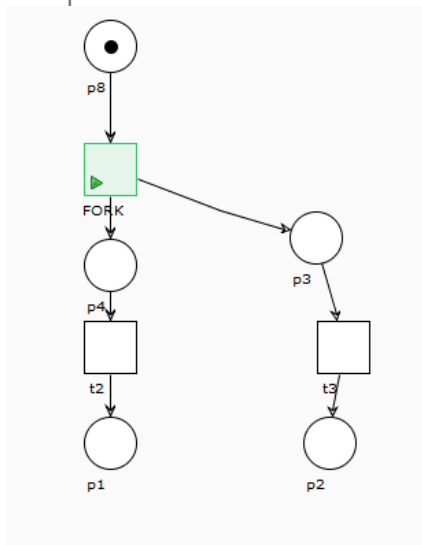
La denominación de FORK viene del nombre de la llamada a sistema que realizaban los procesos de los sistemas operativos Unix para la creación de nuevos procesos



1.18. Fork o creación de procesos

Creación de procesos o Fork

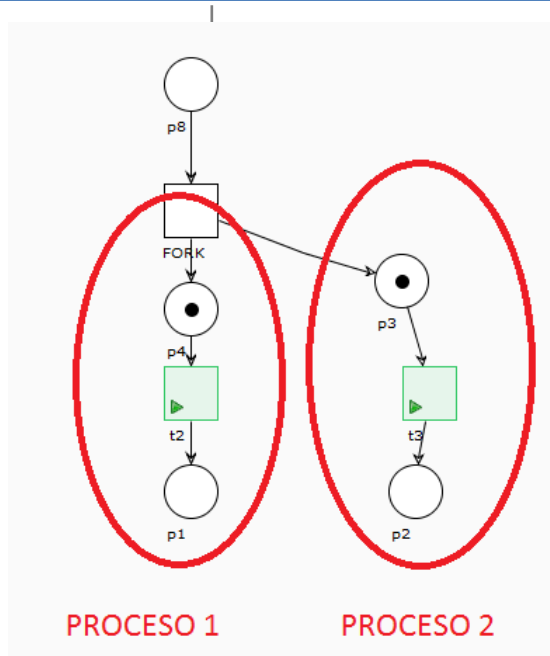
Veamos más en detalle la creación del nuevo proceso. A partir del estado 1 del Proceso 1, somos capaces de activar la transacción FORK que dará como resultado un token nuevo en cada uno de los estados destino.



1.19. Fork listo para ejecución

Creación de procesos o Fork

A partir del momento en que hemos ejecutado nuestro FORK tenemos ya dos procesos simultáneos ejecutándose en nuestro sistema, cada uno independiente del otro.

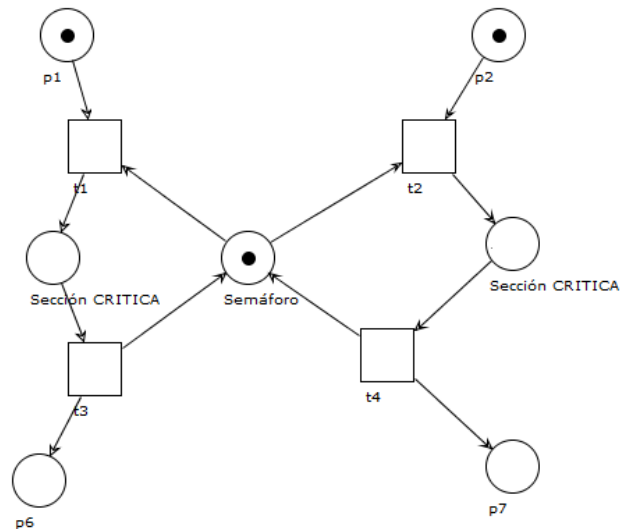


ursos gráficos y/o
omplementarios

1.20. Fork o creación de procesos ejecutado

Secciones críticas y exclusión mutua

Cuando tenemos más de un proceso ejecutándose en paralelo y que comparten datos nos podemos encontrar con el problema de qué esos **datos tengan que ser accedidos de manera simultánea** por varios procesos. Si el acceso es el modo lectura por parte de todos los procesos no hay problema pero las interferencias serias comienzan cuando uno de los procesos desea modificar esos datos.



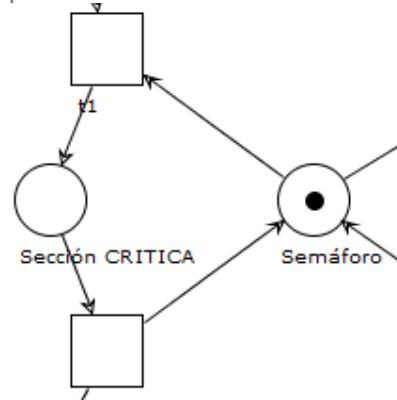
1.21. Red Petri para una exclusión mutua

Secciones críticas y exclusión mutua

Este escenario puede llevar a que un proceso esté leyendo los datos y justo en ese momento un segundo proceso los modifique de manera que el proceso lector puede estar leyendo datos erróneos o datos no actualizados completamente.

De este modo esta situación es la que tenemos que evitar a toda costa, y la zona de código donde cada uno de los procesos modifican estos datos es la llamada **Sección Crítica del proceso**.

Zona reservada para Recursos gráficos y/o contenidos teóricos complementarios

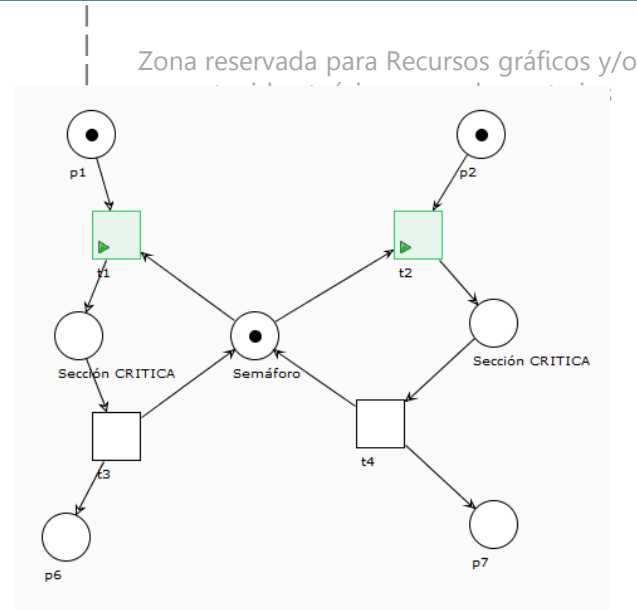


1.22. Sección crítica del proceso

Secciones críticas y exclusión mutua

Una solución a este problema es la denominada zona de exclusión mutua es decir crear una sección crítica donde solo puede haber un proceso ejecutándose a la vez y en esa sección sea donde se accede a esos datos conflictivos.

El acceso a las secciones críticas o de exclusión mutua se suelen regular mediante estructuras denominadas semáforos y que podemos modelar en redes de Petri de la manera que vemos en la figura. Este es el momento previo al intento de entrada de dos (o más) procesos a sus respectivas secciones críticas. Vemos como se activan las transiciones de entrada, pero sólo una se podrá ejecutar, ya que ambas necesitan el token del Semáforo para poderse disparar, como veremos en la siguiente figura entrará el proceso que primero dispare la transición de entrada y "robe" el token.



1.23. Acceso a la sección crítica del proceso

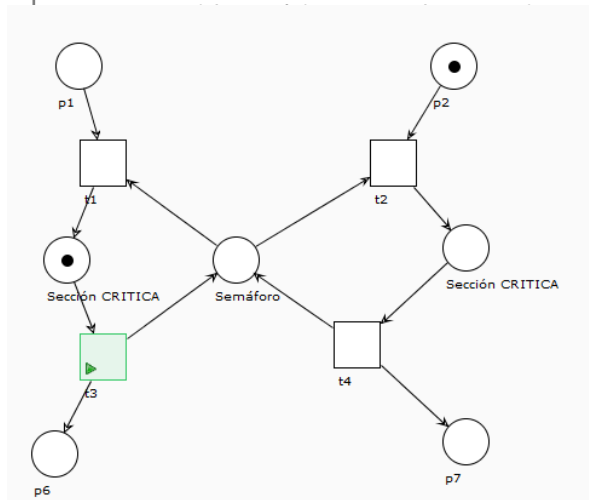
Secciones críticas y exclusión mutua

El acceso a la sección crítica en este ejemplo lo ha ganado el Proceso 1 (izquierdo), al ejecutar su transición y consumir el token del semáforo.

Ahora el proceso 1 podrá tratar de manera conveniente los datos compartidos sin el problema de que el proceso 2 esté de manera simultánea accediendo a estos mismos datos compartidos, ya que el proceso 2 está esperando a disparar su transición t2, ya que necesita del token del Semáforo que ya no está disponible.

La única evolución posible del sistema como vemos es que el proceso 1 termine de tratar los datos compartidos y salga de su sección crítica, devolviendo el token al semáforo a través de la transición t3.

Zona reservada para Recursos gráficos y/o



1.24. Dentro de la sección crítica del proceso

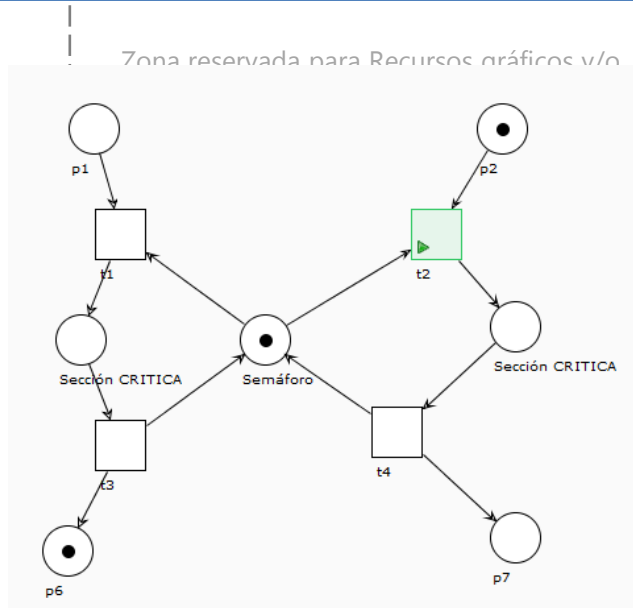
Secciones críticas y exclusión mutua

La salida de la sección crítica

Como hemos visto en la anterior figura, al salir se devuelve el token al semáforo, por lo que se habilita de nuevo la transición t_2 del proceso 2, pudiendo ahora de nuevo el proceso 2 entrar en su sección crítica.

En este caso ya no hay otra competencia para entrar en la Sección Crítica, por lo que el Proceso 2 podrá entrar en la sección crítica y tratar los datos del modo en que precise.

Hemos conseguido así el acceso ordenado y exclusivo a los datos compartidos.

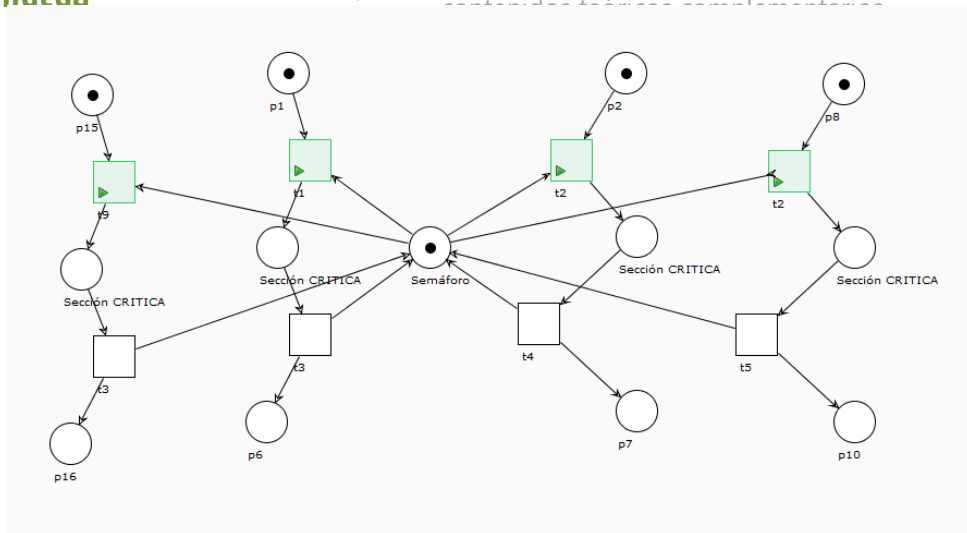


1.25. Salida de la sección crítica del proceso

Secciones críticas y exclusión mutua

Sección crítica para n procesos

La extrapolación de este modelo de Red de Petri a n procesos es en realidad muy sencilla, simplemente hay que duplicar el modelo del proceso conectando todas las transiciones de entrada y salida de los n procesos al semáforo compartido, de modo que un único token en el semáforo seguirá asegurando que un único proceso será el que esté de manera exclusiva en todo momento accediendo a los datos compartidos

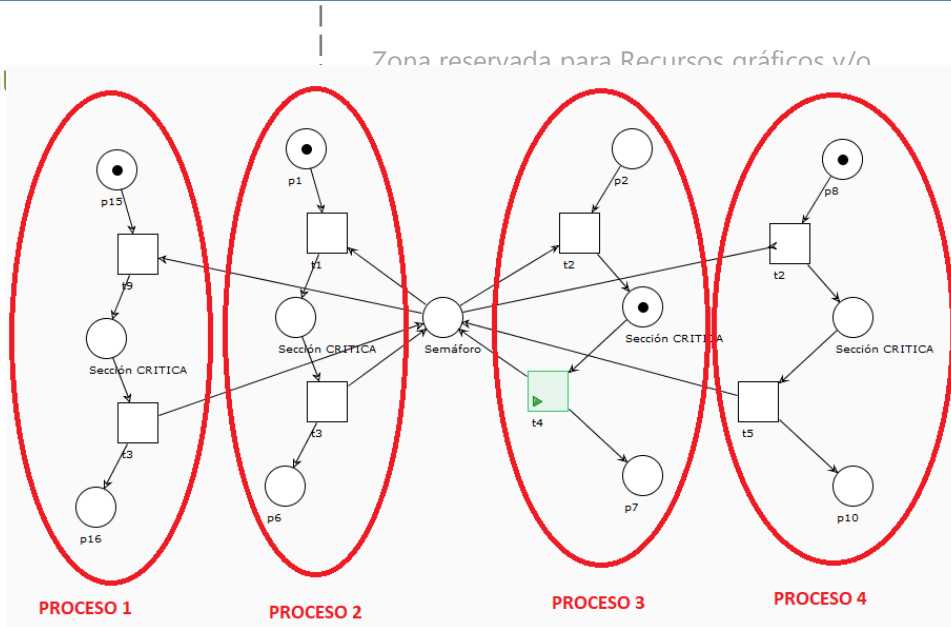


1.26. Sección crítica multi proceso

Secciones críticas y exclusión mutua

De este modo podemos ver como el **acceso a las secciones críticas** funciona exactamente igual que en el modelo anteriormente descrito.

El proceso 3 es el que en este momento ha conseguido el acceso a su sección crítica y ha bloqueado al resto de procesos que quedan a la espera de que salga y libere el token del semáforo para que pueda acceder el siguiente proceso, que en igualdad de condiciones será el **Scheduler** el que elija para ejecutar.



1.27. Sección crítica ganada por el proceso 3